



US 20250284424A1

(19) **United States**

(12) **Patent Application Publication**
Curtis-Maury et al.

(10) **Pub. No.: US 2025/0284424 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **SPACE BALANCING WITHIN A DISTRIBUTED STORAGE SYSTEM USING DISAGGREGATED STORAGE**

(71) Applicant: **NetApp, Inc.**, San Jose, CA (US)

(72) Inventors: **Matthew Curtis-Maury**, Apex, NC (US); **Yash Hetal Trivedi**, San Jose, CA (US); **Travis Ryan Grusecki**, Santa Clara, CA (US); **Bryan Joshua Prosser**, Durham, NC (US); **Abdul Basit**, Morrisville, NC (US); **Sushrut Bhowmik**, Cary, NC (US); **Travis Ian Callahan**, Apex, NC (US); **Anil Paul Thoppil**, Pleasanton, CA (US); **Ananthan Subramanian**, San Ramon, CA (US)

(73) Assignee: **NetApp, Inc.**, San Jose, CA (US)

(21) Appl. No.: **19/068,324**

(22) Filed: **Mar. 3, 2025**

Related U.S. Application Data

(63) Continuation-in-part of application No. 18/595,785, filed on Mar. 5, 2024.

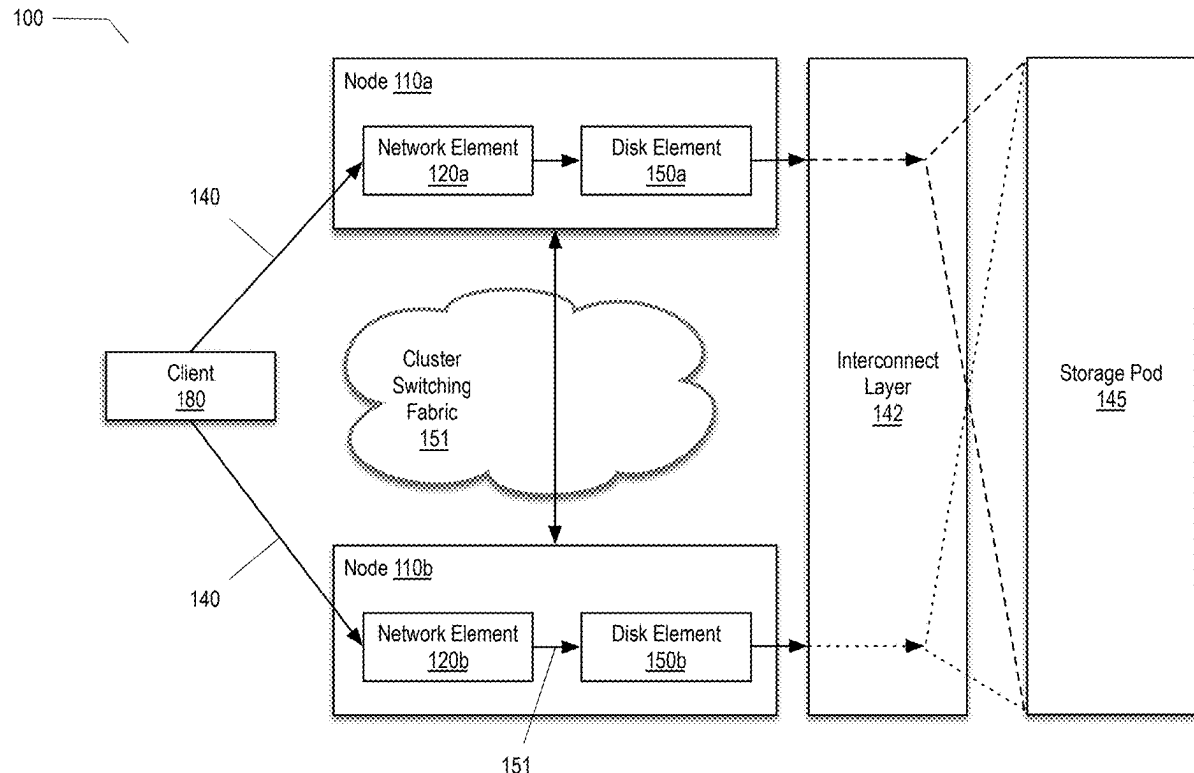
Publication Classification

(51) **Int. Cl.**
G06F 3/06 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 3/0644** (2013.01); **G06F 3/0604** (2013.01); **G06F 3/0689** (2013.01)

(57) **ABSTRACT**

Systems and methods for transferring ownership of allocation areas (AA) and associated metadata information to facilitate space balancing within a distributed storage system that makes use of disaggregated storage are provided. In various examples described herein, the unit of storage space assignment used to associate blocks of storage space within a storage pod with a given dynamically extensible file system (DEFS) is an AA, representing a large chunk of one or more gigabytes (GB). The use of AAs described herein allows disk space to be used more elastically across individual nodes of a storage cluster. When partial AAs are transferred among DEFSs, the transfer of associated metadata information may make use of various optimizations to minimize the amount of data copying performed. For example, copying physical volume block numbers (PVBNs) directly or indirectly indicative of locations of metafile data is faster than copying the underlying metafile data.



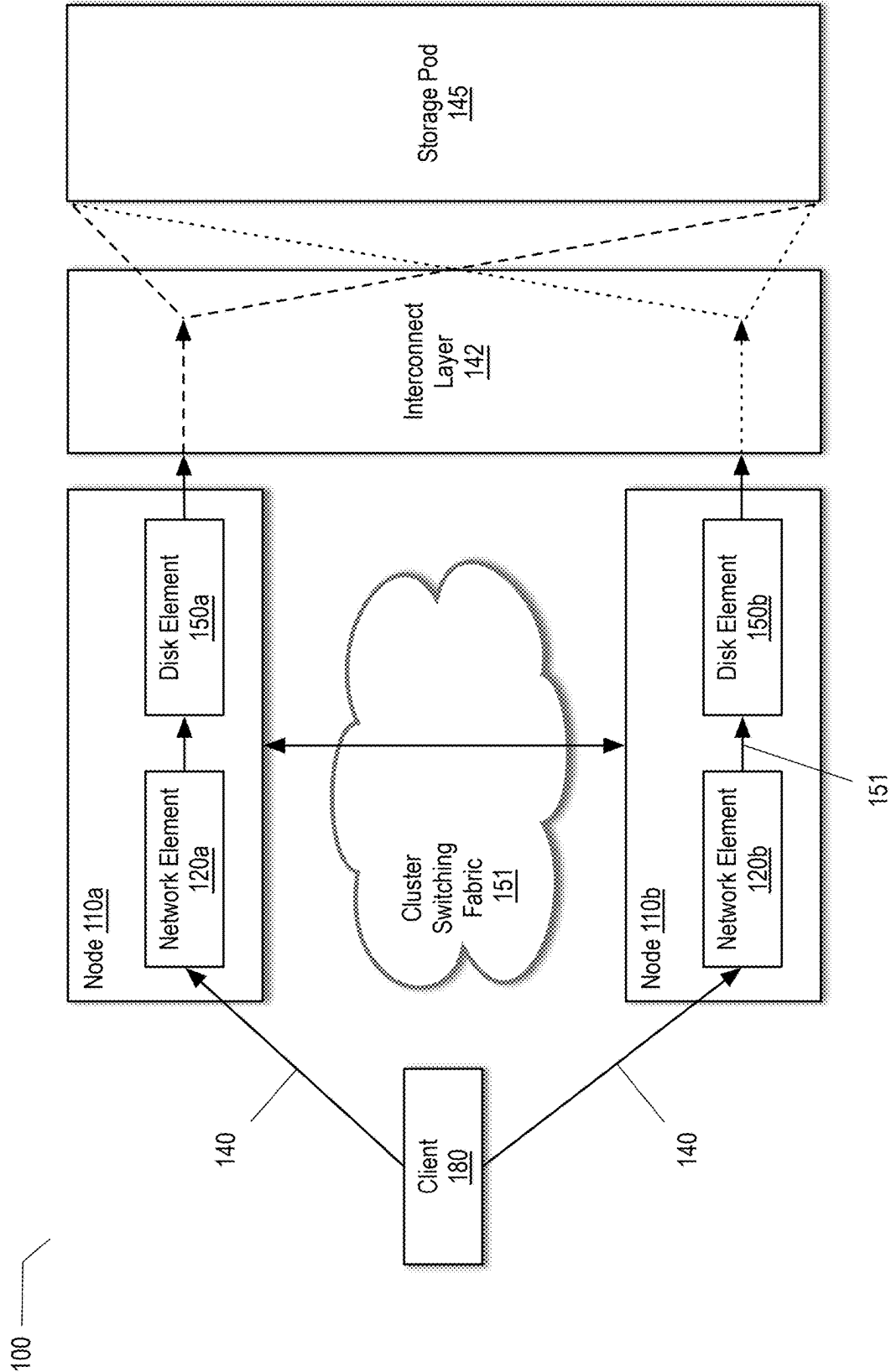


FIG. 1

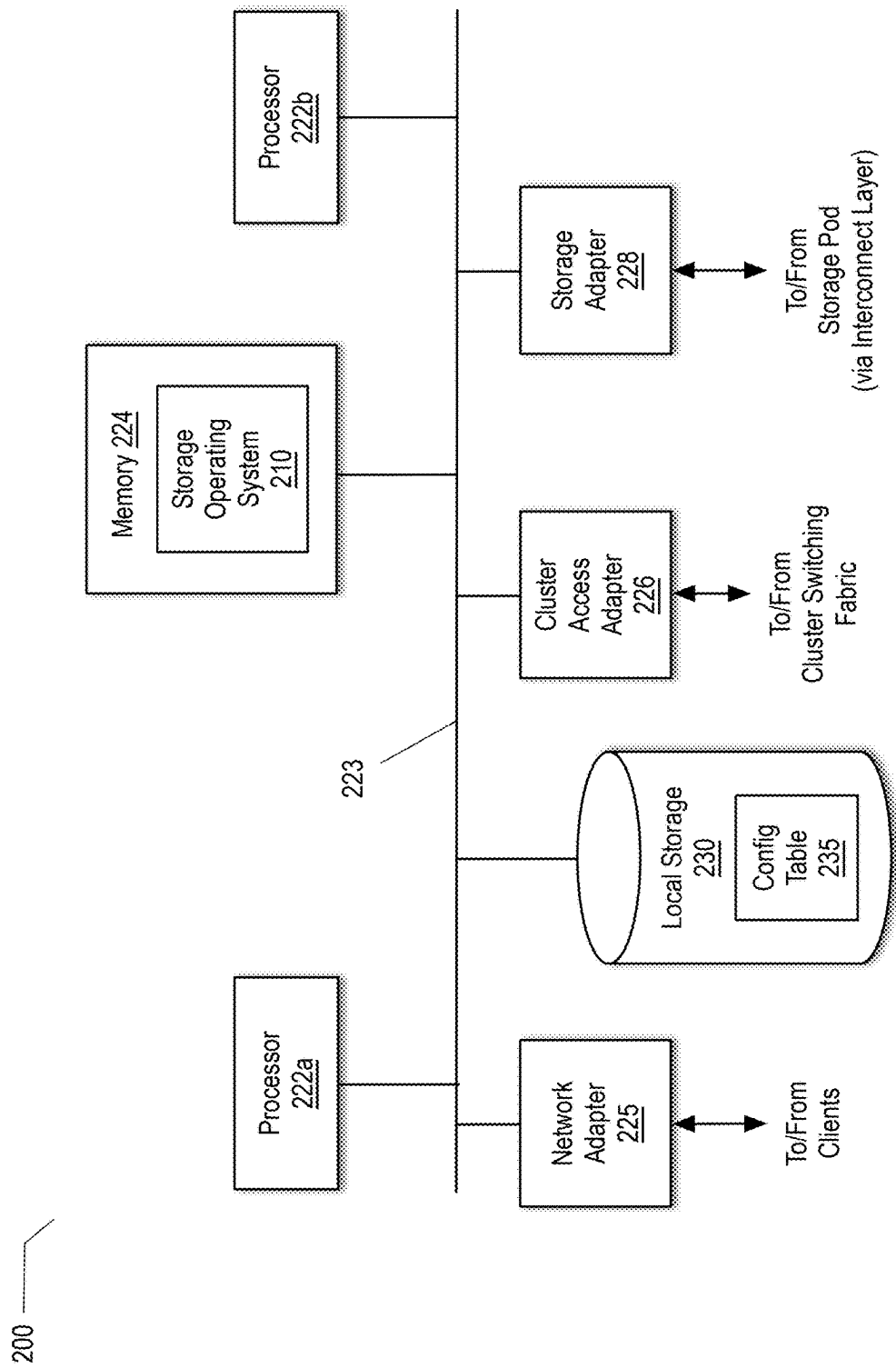


FIG. 2

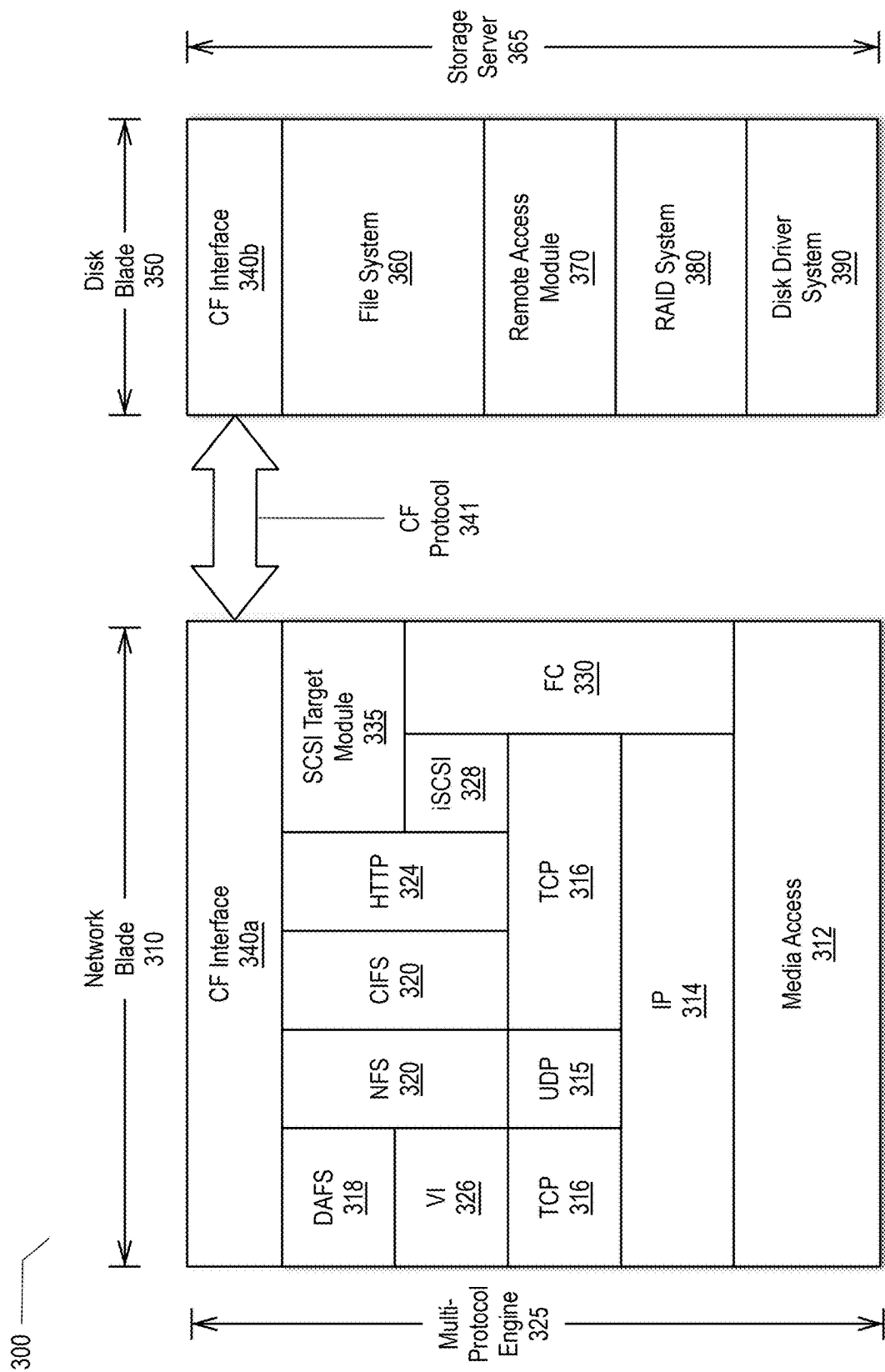


FIG. 3

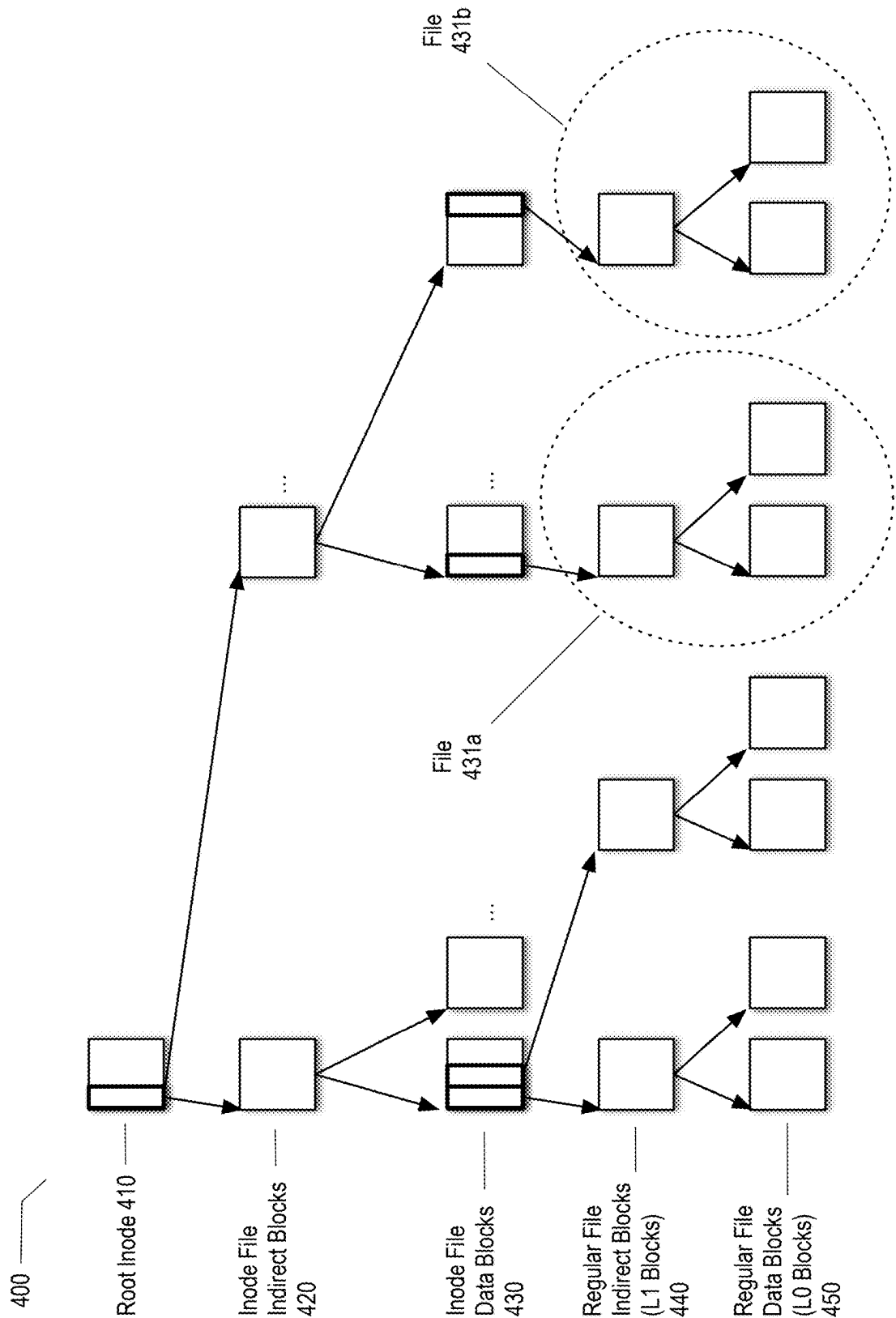


FIG. 4

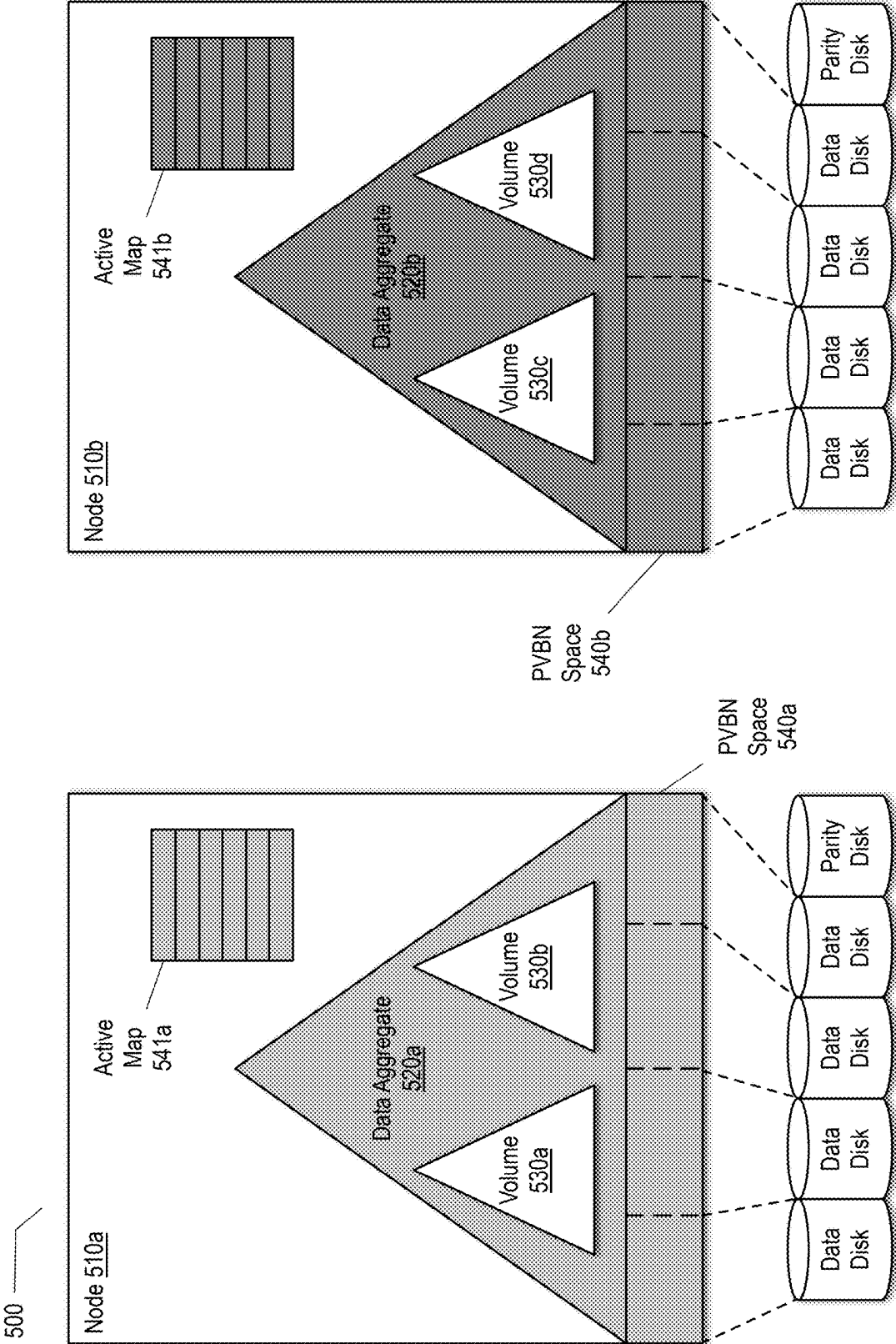


FIG. 5 (Prior Art)

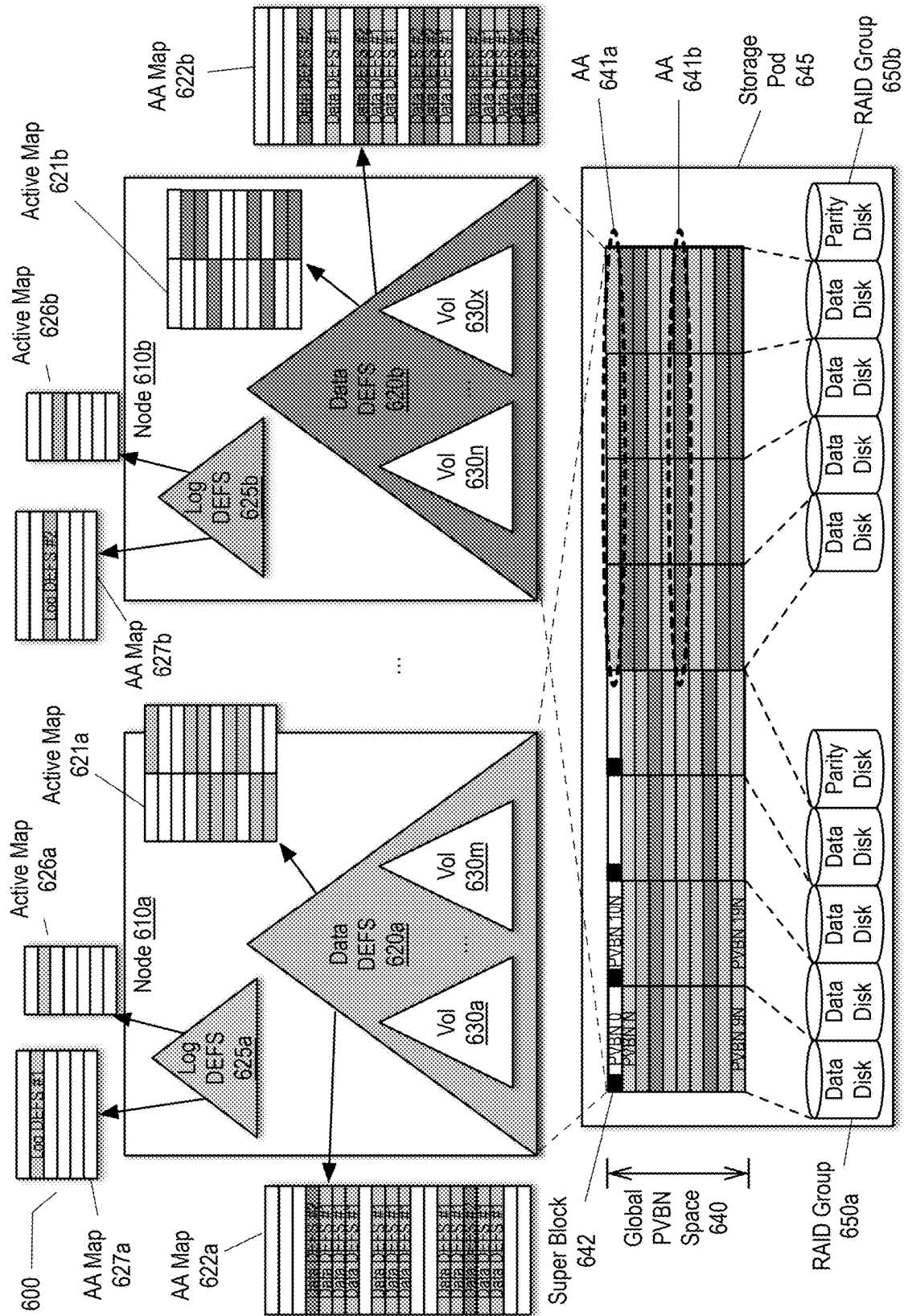


FIG. 6A

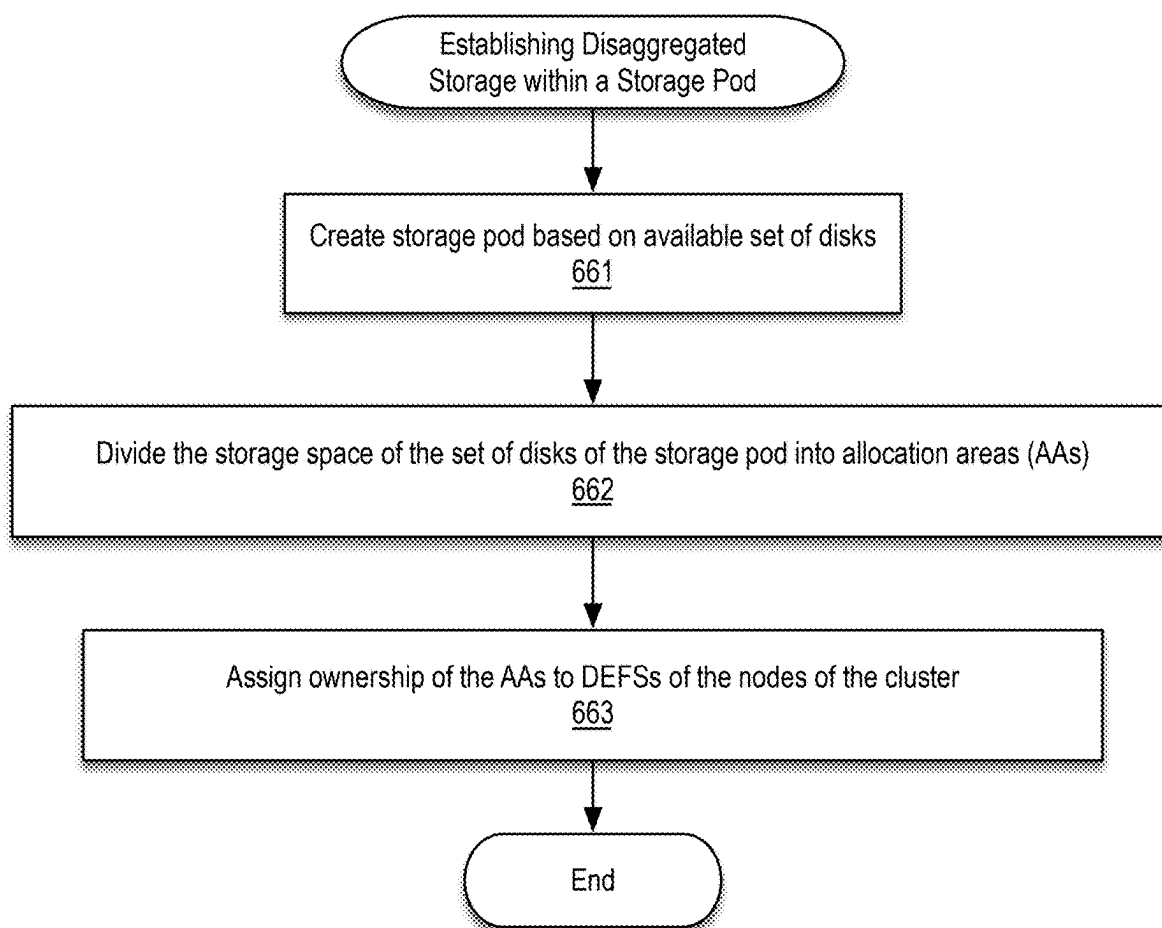


FIG. 6B

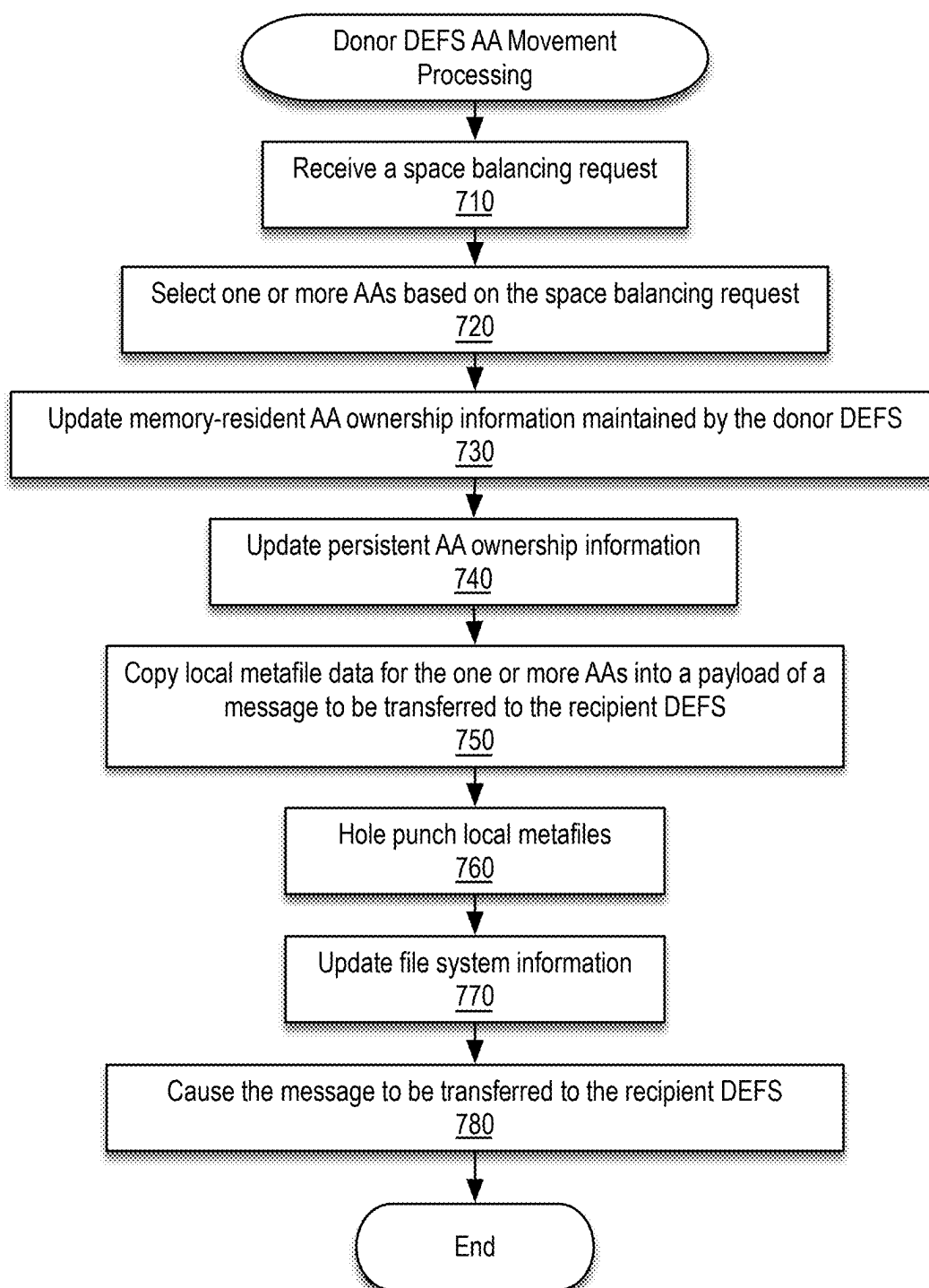


FIG. 7

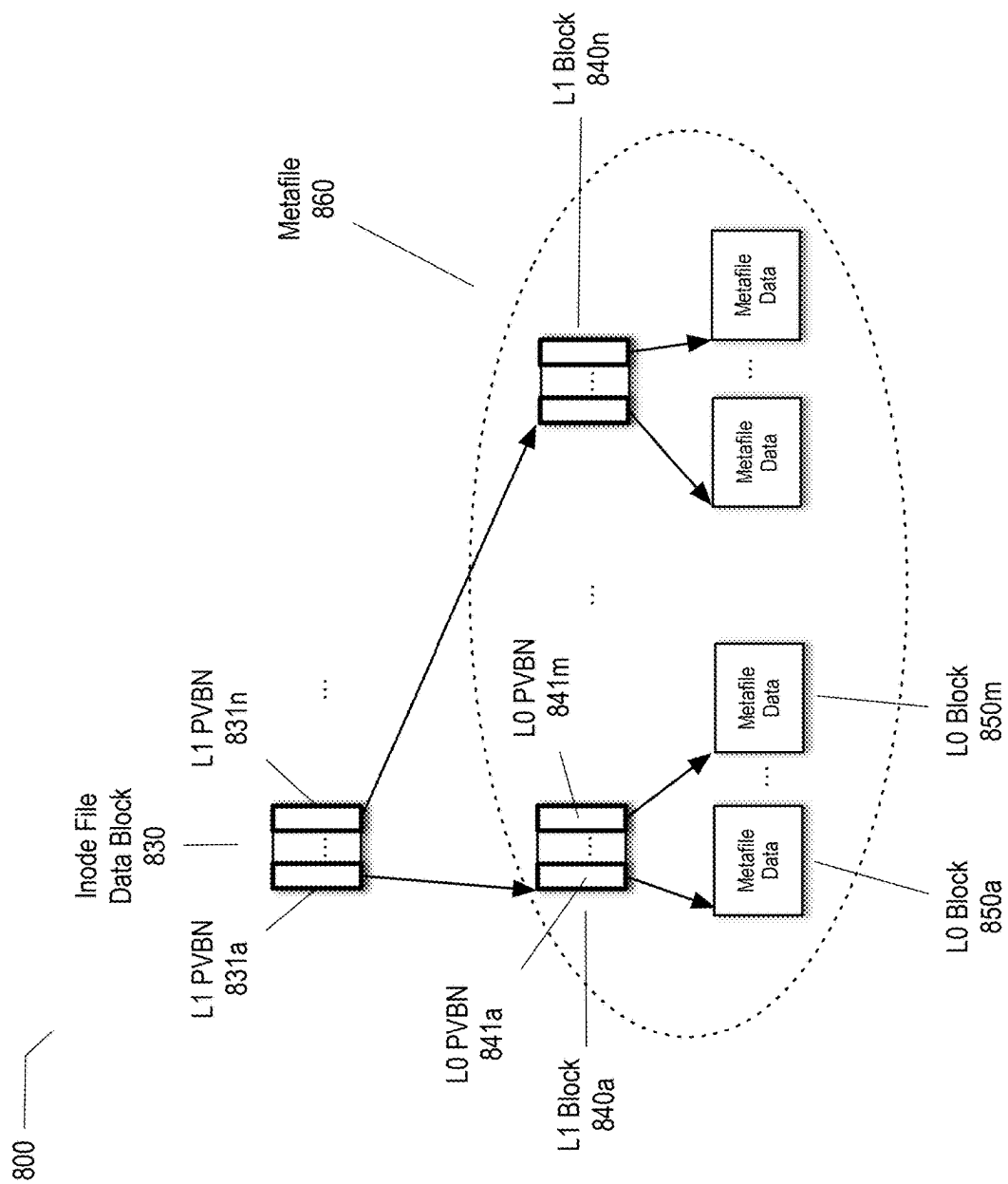


FIG. 8

900

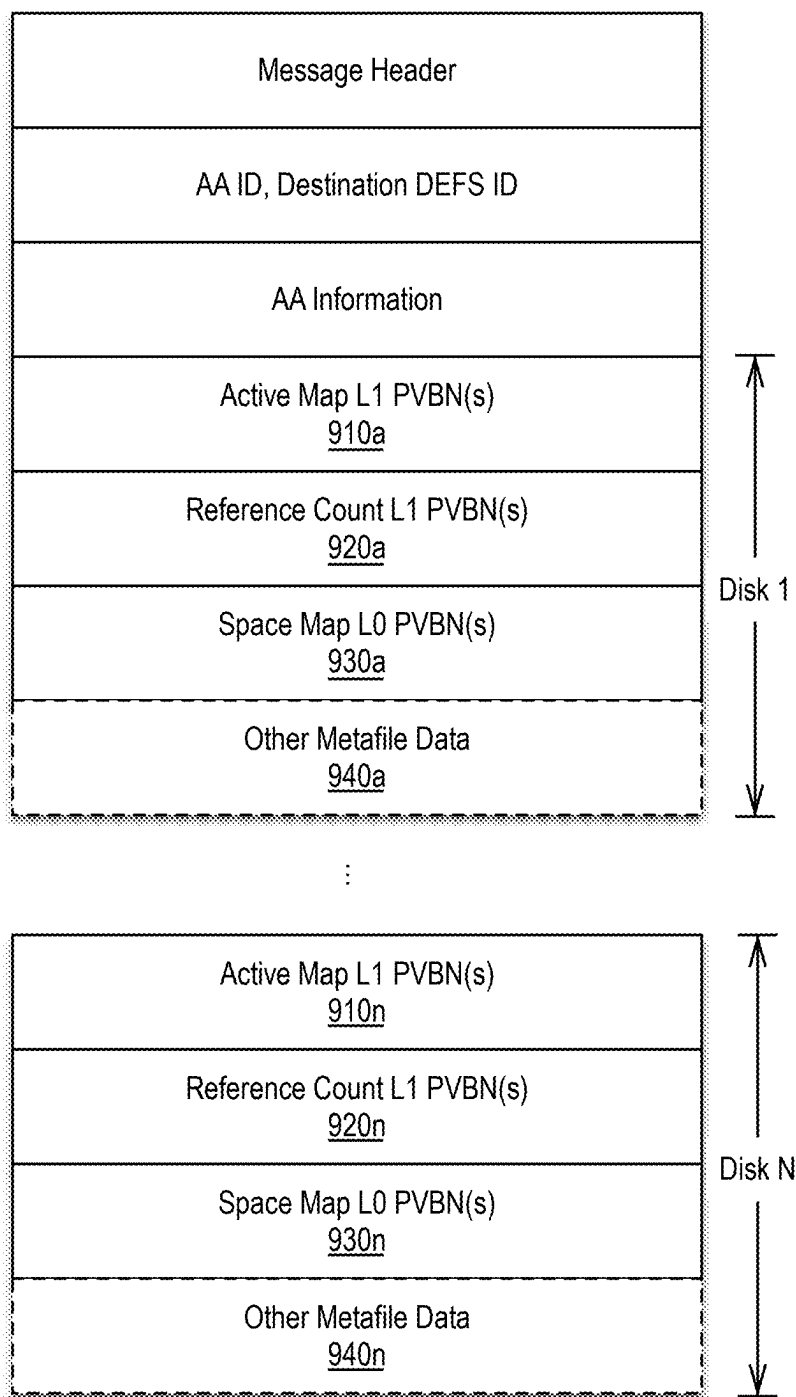
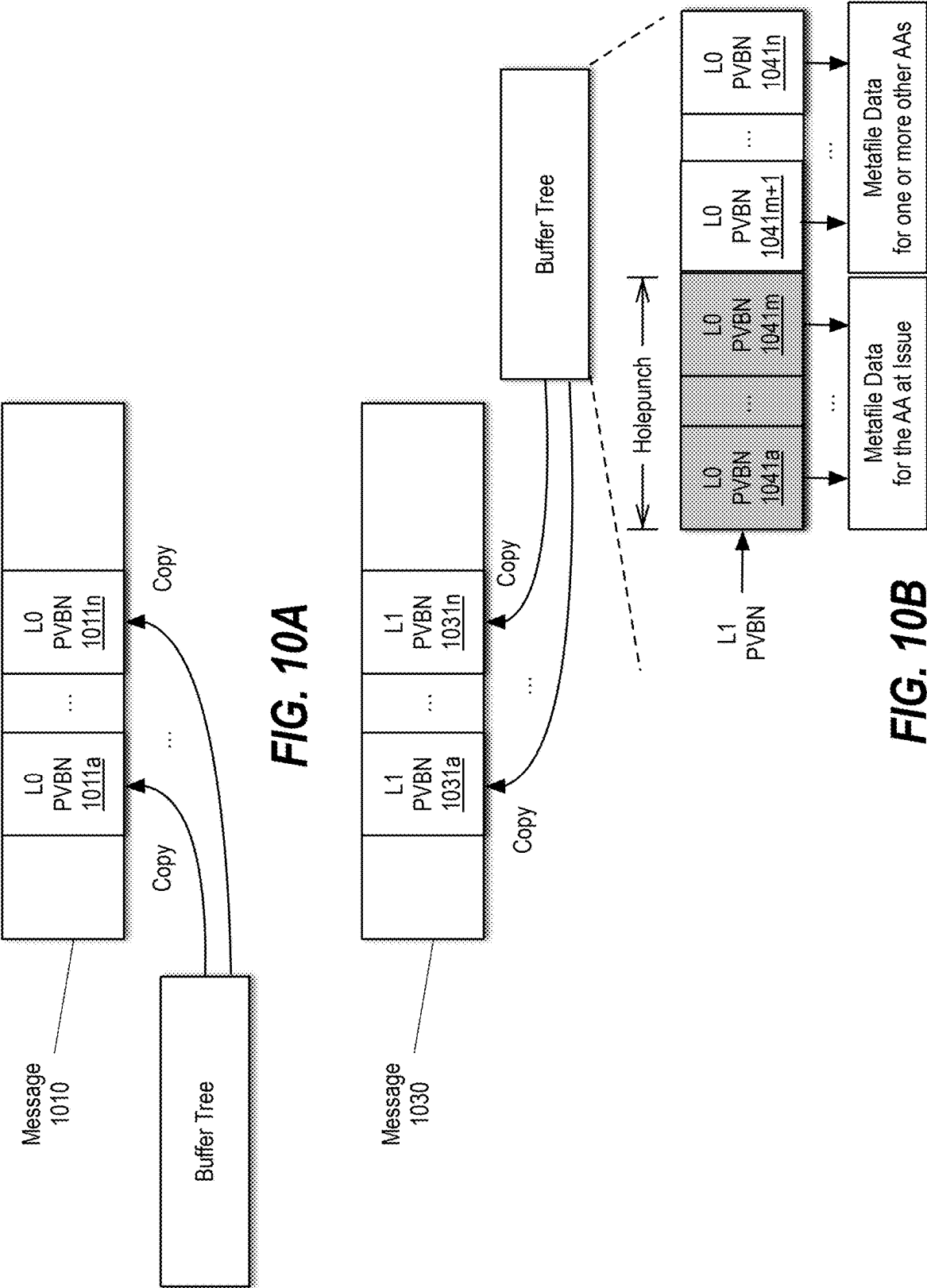


FIG. 9



SPACE BALANCING WITHIN A DISTRIBUTED STORAGE SYSTEM USING DISAGGREGATED STORAGE

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is a continuation-in-part of U.S. patent application Ser. No. 18/595,785, filed on Mar. 5, 2024, which is hereby incorporated by reference in its entirety for all purposes.

BACKGROUND

Field

[0002] Various embodiments of the present disclosure generally relate to storage systems. In particular, some embodiments relate to the implementation and use of disaggregated storage space of a storage pod by a distributed storage system having a disaggregated storage architecture to, among other things, avoid silos of storage space associated with a given node of the cluster and allow disk space to be used more fluidly/elastically across all the individual storage systems (e.g., nodes) of a distributed storage system by providing efficient mechanisms for managing file system metadata associated with allocation areas into which the storage pod is partitioned, for example, as part of space balancing operations.

Description of the Related Art

[0003] Distributed storage systems generally take the form of a cluster of storage controllers (or nodes in virtual or physical form). As a result of sub-optimal infrastructure architectures, prior scale-out storage solutions do not effectively utilize all three vectors of infrastructure (i.e., compute, network, and storage). For example, as shown in FIG. 5, each node of a distributed storage system may be associated with a dedicated pool of storage space (e.g., a node-level aggregate representing a file system that holds one or more volumes created over one or more RAID groups and which is only accessible from a single node at a time), thereby creating storage silos.

SUMMARY

[0004] Systems and methods are described for transferring ownership of allocation areas and associated metadata information to facilitate space balancing within a distributed storage system that makes use of disaggregated storage. A storage pod having a group of storage devices containing multiple Redundant Array of Independent Disks (RAID) groups is provided in which an entirety of a global physical volume block number (PVCN) space associated with the storage pod is accessible to all nodes of multiple nodes of a cluster representing a distributed storage system via their respective dynamically extensible file systems (DEFSs). Storage space associated with the group of storage devices is partitioned into multiple allocation areas (AAs), in which a given AA is owned by a given DEFS of multiple DEFSs of the cluster. A space balancing request is received by a donor DEFS of a first node of the multiple nodes, indicating (i) a number of AAs to be transferred to a recipient DEFS of a second node of the multiple nodes and (ii) a quality of the AAs to be transferred. A set of one or more AAs currently owned by the donor DEFS is selected by the first node that

satisfy the space balancing request. Ownership of the set of AAs and metadata information associated with the set of AAs is transferred by the first node to the recipient DEFS. [0005] Other features of embodiments of the present disclosure will be apparent from accompanying drawings and detailed description that follows.

BRIEF DESCRIPTION OF THE DRAWINGS

[0006] In the Figures, similar components and/or features may have the same reference label. Further, various components of the same type may be distinguished by following the reference label with a second label that distinguishes among the similar components. If only the first reference label is used in the specification, the description is applicable to any one of the similar components having the same first reference label irrespective of the second reference label.

[0007] FIG. 1 is a block diagram illustrating a plurality of nodes interconnected as a cluster in accordance with an embodiment of the present disclosure.

[0008] FIG. 2 is a block diagram illustrating a node in accordance with an embodiment of the present disclosure.

[0009] FIG. 3 is a block diagram illustrating a storage operating system in accordance with an embodiment of the present disclosure.

[0010] FIG. 4 is a block diagram illustrating a tree of blocks representing of an example a file system layout in accordance with an embodiment of the present disclosure.

[0011] FIG. 5 is a block diagram illustrating a distributed storage system architecture in which the entirety of a given storage device and a given RAID group are owned by an aggregate and the aggregate file system is only visible from one node, thereby resulting in silos of storage space.

[0012] FIG. 6A is a block diagram illustrating a distributed storage system architecture that provides disaggregated storage in accordance with an embodiment of the present disclosure.

[0013] FIG. 6B is a high-level flow diagram illustrating operations for establishing disaggregated storage within a storage pod in accordance with an embodiment of the present disclosure.

[0014] FIG. 7 is a flow diagram illustrating operations for performing AA movement in accordance with an embodiment of the present disclosure.

[0015] FIG. 8 is a block diagram conceptually illustrating a portion of a buffer tree containing information regarding a metafile in accordance with an embodiment of the present disclosure.

[0016] FIG. 9 is a block diagram conceptually illustrating a message format that may be used in connection with performing AA movement in accordance with an embodiment of the present disclosure.

[0017] FIG. 10A is a block diagram conceptually illustrating a first optimization for transferring metafile data in accordance with an embodiment of the present disclosure.

[0018] FIG. 10B is a block diagram conceptually illustrating a second optimization for transferring metafile data in accordance with an embodiment of the present disclosure.

DETAILED DESCRIPTION

[0019] Systems and methods are described for transferring ownership of allocation areas and associated metadata information to facilitate space balancing within a distributed storage system that makes use of disaggregated storage. As

compared to existing scale out storage solution architectures, various examples described herein facilitate various advantages, including, but not limited to, one or more of the following:

- [0020]** Simplified management
- [0021]** No silos of storage space
- [0022]** Independent file system operation on each node of a cluster
- [0023]** Independent scaling of performance and capacity (e.g., the ability to independently add nodes and/or disks on demand)
- [0024]** Reduced internode (or “East-West”) communications/traffic.
- [0025]** No additional redirection in the Input/Output (I/O) path
- [0026]** No additional write amplification
- [0027]** Integration with existing storage operating systems (e.g., the ONTAP data management software available from NetApp, Inc. of San Jose, CA).
- [0028]** Distributed disk operations
- [0029]** The ability to use all disks associated with a distributed storage system in a more uniform manner
- [0030]** In various examples described herein, storage device (or “disk”) used interchangeably throughout this specification) space may be used more fluidly across all the individual storage systems (e.g., nodes) of a distributed storage system (e.g., a cluster of nodes working together), thereby eliminating silos of storage; and processing resource (e.g., central processing unit (CPU)) load may be distributed across the cluster. The proposed architecture seeks to prevent a given disk from being tied to any single node of the cluster by introducing a new construct referred to herein as a “dynamically extensible file system,” examples of which are described further below with reference to FIG. 6A. In contrast to the entirety of a given storage device (e.g., a disk) being owned by a node-level aggregate and the aggregate file system being visible from only one node of a cluster as shown and described with reference to FIG. 5, the use of dynamically extensible file systems facilitates visibility by all nodes in the cluster to the entirety of a global physical volume block number (PVCN) space of the disks associated with a single “storage pod” (another new construct introduced herein) that may be shared by all of the nodes of the cluster with space from the global PVCN space being used on demand.
- [0031]** In one embodiment, each node of a cluster has access to do read and write to all the disks in a storage pod associated with a cluster. Given all the nodes have access to the same disks, a RAID subsystem or layer can now assimilate the same RAID tree from the same set of disks and present the global PVCN space to the file system (e.g., a write anywhere file system, such as the write anywhere file layout (WAFL) file system available from NetApp, Inc. of San Jose, CA). Using the global PVCN space, each node of the cluster can create an independent file system that it needs. As those skilled in the art will appreciate, it would be dangerous for each node to allocate from the same global PVCN space independently and without limitation. As such, examples of the proposed architecture restrict each dynamically extensible file system to use (consume) space only from the blocks assigned to it or “owned” by it. As such, when performing writes, each dynamically extensible file system stays in its own lane without the need for complex access control mechanisms, such as locks.

[0032] As described further below, in some examples, the association of blocks to a dynamically extensible file system may be in large chunks of one or more gigabytes (GB), which are referred to herein as “allocation areas” (AAs) that each include multiple RAID stripes. The use of large, multi-GB chunks, as the unit of space allocation/assignment to dynamically extensible file systems facilitates ease of management (e.g., by way of reducing the frequency of ownership transfers among dynamically extensible file systems) of these AAs. The assignment of AAs to individual dynamically extensible file systems, which in turn are owned by nodes, additionally helps each node do its write allocation independently since, by definition an entire RAID stripe is owned by a single dynamically extensible file system. In some embodiments, dynamically extensible file systems also minimize or at least significantly reduce the need for internode communications. For example, dynamically extensible file systems can limit their coordination across nodes to situations in which space balancing is to be performed (e.g., responsive to a node running low on free storage space relative to the other nodes), which is not a frequent operation. Responsive to a space balancing trigger event, a given dynamically extensible file system (or the node owning given dynamically extensible file system on behalf of the given dynamically extensible file system) may request space be reassigned to it from one or more other dynamically extensible file systems. The combination of visibility into the entire global PVCN space and the use of dynamically extensible file systems and their association with a given portion of the disaggregated storage of a storage pod to which a given dynamically extensible file system has exclusive write access enables each node to run independently most of the time.

[0033] In the following description, numerous specific details are set forth in order to provide a thorough understanding of embodiments of the present disclosure. It will be apparent, however, to one skilled in the art that embodiments of the present disclosure may be practiced without some of these specific details. In other instances, well-known structures and devices are shown in block diagram form.

Terminology

[0034] Brief definitions of terms used throughout this application are given below.

[0035] The terms “connected” or “coupled” and related terms are used in an operational sense and are not necessarily limited to a direct connection or coupling. Thus, for example, two devices may be coupled directly, or via one or more intermediary media or devices. As another example, devices may be coupled in such a way that information can be passed there between, while not sharing any physical connection with one another. Based on the disclosure provided herein, one of ordinary skill in the art will appreciate a variety of ways in which connection or coupling exists in accordance with the aforementioned definition.

[0036] If the specification states a component or feature “may”, “can”, “could”, or “might” be included or have a characteristic, that particular component or feature is not required to be included or have the characteristic.

[0037] The terms “component”, “module”, “system,” and the like as used herein are intended to refer to a computer-related entity, either software-executing general-purpose processor, hardware, firmware and a combination thereof. For example, a component may be, but is not limited to

being, a process running on a hardware processor, a hardware processor, an object, an executable, a thread of execution, a program, and/or a computer. By way of illustration, both an application running on a server and the server can be a component. One or more components may reside within a process and/or thread of execution, and a component may be localized on one computer and/or distributed between two or more computers. Also, these components can be executed from various computer readable media having various data structures stored thereon. The components may communicate via local and/or remote processes such as in accordance with a signal having one or more data packets (e.g., data from one component interacting with another component in a local system, distributed system, and/or across a network such as the Internet with other systems via the signal).

[0038] The term file/files as used herein include data container/data containers, directory/directories, and/or data object/data objects with structured or unstructured data.

[0039] As used in the description herein and throughout the claims that follow, the meaning of “a,” “an,” and “the” includes plural reference unless the context clearly dictates otherwise. Also, as used in the description herein, the meaning of “in” includes “in” and “on” unless the context clearly dictates otherwise.

[0040] The phrases “in an embodiment,” “according to one embodiment,” and the like generally mean the particular feature, structure, or characteristic following the phrase is included in at least one embodiment of the present disclosure and may be included in more than one embodiment of the present disclosure. Importantly, such phrases do not necessarily refer to the same embodiment.

[0041] As used herein a “cloud” or “cloud environment” broadly and generally refers to a platform through which cloud computing may be delivered via a public network (e.g., the Internet) and/or a private network. The National Institute of Standards and Technology (NIST) defines cloud computing as “a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.” P. Mell, T. Grance, The NIST Definition of Cloud Computing, National Institute of Standards and Technology, USA, 2011. The infrastructure of a cloud may be deployed in accordance with various deployment models, including private cloud, community cloud, public cloud, and hybrid cloud. In the private cloud deployment model, the cloud infrastructure is provisioned for exclusive use by a single organization comprising multiple consumers (e.g., business units), may be owned, managed, and operated by the organization, a third party, or some combination of them, and may exist on or off premises. In the community cloud deployment model, the cloud infrastructure is provisioned for exclusive use by a specific community of consumers from organizations that have shared concerns (e.g., mission, security requirements, policy, and compliance considerations), may be owned, managed, and operated by one or more of the organizations in the community, a third party, or some combination of them, and may exist on or off premises. In the public cloud deployment model, the cloud infrastructure is provisioned for open use by the general public, may be owned, managed, and operated by a cloud provider or hyperscaler (e.g., a business, academic, or government organization, or some combination of them), and exists on the

premises of the cloud provider. The cloud service provider may offer a cloud-based platform, infrastructure, application, or storage services as-a-service, in accordance with a number of service models, including Software-as-a-Service (SaaS), Platform-as-a-Service (PaaS), and/or Infrastructure-as-a-Service (IaaS). In the hybrid cloud deployment model, the cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public) that remain unique entities, but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load balancing between clouds).

[0042] As used herein, a “storage system” or “storage appliance” generally refers to a type of computing appliance or node, in virtual or physical form, that provides data to, or manages data for, other computing devices or clients (e.g., applications). The storage system may be part of a cluster of multiple nodes representing a distributed storage system. In various examples described herein, a storage system may be run (e.g., on a VM or as a containerized instance, as the case may be) within a public cloud provider.

[0043] As used herein, the term “storage operating system” generally refers to computer-executable code operable on a computer to perform a storage function that manages data access and may, in the case of a storage system (e.g., a node), implement data access semantics of a general purpose operating system. The storage operating system can also be implemented as a microkernel, an application program operating over a general-purpose operating system, such as UNIX or Windows NT, or as a general-purpose operating system with configurable functionality, which is configured for storage applications as described herein. In some embodiments, a light-weight data adaptor may be deployed on one or more server or compute nodes added to a cluster to allow compute-intensive data services to be performed without adversely impacting performance of storage operations being performed by other nodes of the cluster. The light-weight data adaptor may be created based on a storage operating system but, since the server node will not participate in handling storage operations on behalf of clients, the light-weight data adaptor may exclude various subsystems/modules that are used solely for serving storage requests and that are unnecessary for performance of data services. In this manner, compute intensive data services may be handled within the cluster by one of more dedicated compute nodes.

[0044] As used herein, a “cloud volume” generally refers to persistent storage that is accessible to a virtual storage system by virtue of the persistent storage being associated with a compute instance in which the virtual storage system is running. A cloud volume may represent a hard-disk drive (HDD) or a solid-state drive (SSD) from a pool of storage devices within a cloud environment that is connected to the compute instance through Ethernet or fibre channel (FC) switches as is the case for network-attached storage (NAS) or a storage area network (SAN). Non-limiting examples of cloud volumes include various types of SSD volumes (e.g., AWS Elastic Block Store (EBS) gp2, gp3, io1, and io2 volumes for EC2 instances) and various types of HDD volumes (e.g., AWS EBS st1 and sc1 volumes for EC2 instances).

[0045] As used herein a “consistency point” or “CP” generally refers to the act of writing data to disk and updating active file system pointers. In various examples, when a file system of a storage system receives a write

request, it commits the data to permanent storage before the request is confirmed to the writer. Otherwise, if the storage system were to experience a failure with data only in volatile memory, that data would be lost, and underlying file structures could become corrupted. Physical storage appliances commonly use battery-backed high-speed non-volatile random access memory (NVRAM) as a journaling storage media to journal writes and accelerate write performance while providing permanence, because writing to memory is much faster than writing to storage (e.g., disk). Storage systems may also implement a buffer cache in the form of an in-memory cache to cache data that is read from data storage media (e.g., local mass storage devices or a storage array associated with the storage system) as well as data modified by write requests. In this manner, in the event a subsequent access relates to data residing within the buffer cache, the data can be served from local, high performance, low latency storage, thereby improving overall performance of the storage system. Virtual storage appliances may use NV storage backed by cloud volumes in place of NVRAM for journaling storage and for the buffer cache. Regardless of whether NVRAM or NV storage is utilized, the modified data may be periodically (e.g., every few seconds) flushed to the data storage media. As the buffer cache may be limited in size, an additional cache level may be provided by a victim cache, typically implemented within a slower memory or storage device than utilized by the buffer cache, that stores data evicted from the buffer cache. The event of saving the modified data to the mass storage devices may be referred to as a CP. At a CP, the file system may save any data that was modified by write requests to persistent data storage media. As will be appreciated, when using a buffer cache, there is a small risk of a system failure occurring between CPs, causing the loss of data modified after the last CP. Consequently, the storage system may maintain an operation log or journal of certain storage operations within the journaling storage media that have been performed since the last CP. This log may include a separate journal entry (e.g., including an operation header) for each storage request received from a client that results in a modification to the file system or data. Such entries for a given file may include, for example, “Create File,” “Write File Data,” and the like. Depending upon the operating mode or configuration of the storage system, each journal entry may also include the data to be written according to the corresponding request. The journal may be used in the event of a failure to recover data that would otherwise be lost. For example, in the event of a failure, it may be possible to replay the journal to reconstruct the current state of stored data just prior to the failure. As described further below, in various examples there may be one or more predefined or configurable triggers (CP triggers). Responsive to a given CP trigger (or at a CP), the file system may save any data that was modified by write requests to persistent data storage media.

[0046] As used herein, a “RAID stripe” generally refers to a set of blocks spread across multiple storage devices (e.g., disks of a disk array, disks of a disk shelf, or cloud volumes) to form a parity group (or RAID group).

[0047] As used herein, an “allocation area” or “AA” generally refers to a group of RAID stripes. In various examples described herein a single storage pod may be shared by a distributed storage system by assigning ownership of AAs to respective dynamically extensible file systems (DEFSs) of a storage system.

[0048] As used herein, “ownership” of an AA generally refers to the ability of the owning DEFS to use the AA space (e.g., the blocks associated with the AA) for performance of writes or write operations. In the context of various embodiments described herein, only one DEFS can write to a given block (PVCN) at a time for multiple correctness reasons, so it is the DEFS that owns the given AA of which the given block is associated that has the exclusive ability among all other DEFSs in the storage system to write to the given block. Further, in embodiments described herein, for the file system metadata to be correct, the file system metadata for a given AA is coordinated in one place.

[0049] As used herein, “space balancing” generally refers to the movement of one or more AAs from one DEFS (which may be referred to as a donor DEFS) to another DEFS (which may be referred to as a recipient DEFS) of a storage system; or stated another way changing of the ownership of one or more AA from the donor DEFS to the recipient DEFS. Space balancing may be performed to address a number of storage space-related issues including, but not limited to, balancing of (i) free space within DEFSs of a storage cluster, (ii) used space within the DEFSs, (iii) total owned space, and/or (iv) AA quality owned by the DEFSs.

[0050] As used herein, a “quality” of an AA generally refers to one of a multiple categories, buckets, bins, or enumerated types of AAs, for example, with respect to the level of usage of PVCNs associated with the AA. In one example, AAs may be categorized coarsely as (i) free AAs, (ii) partial AAs, and (iii) full AAs. In other examples, the partial AAs may be further refined by bucketing or binning the AAs in accordance with predetermined or configurable used space percentage ranges or bands (e.g., of 5 to 10 percent) based on their respective PVCNs that are in use.

[0051] As used herein, a “free allocation area,” an “empty allocation area,” a “free AA,” or an “empty AA” generally refers to an AA in which no PVCNs of the AA are marked as used or in use (containing valid data), for example, by any active maps of a given dynamically extensible file system.

[0052] As used herein, a “partial allocation area” or “partial AA” generally refers to an AA in which one or more PVCNs of the AA are marked as used or in use (containing valid data), for example, by an active map of a given dynamically extensible file system. As discussed further below, in connection with space balancing, while it is preferable to perform AA ownership changes of free AAs, in various examples, space balancing may involve one dynamically extensible file system donating one or more partial AAs to another dynamically extensible file system. In such cases, the additional cost of transferring or copying all or portions of one or more associated metafiles or data structures (e.g., bit maps, such as an active map, a refcount map, a summary map, an AA information map, and a space map) relating to storage space information may be incurred. No such additional cost is incurred when moving or changing ownership of free AAs. These associated metafiles or data structures may, among other things, track which PVCNs are in use, track PVCN counts per AA (e.g., total used blocks and shared references to blocks) and other flags. In some examples, the metafiles associated with storage may be very large. As such, in order to reduce the size of a message (which may also be referred to as an AA package) transferred from the donor DEFS to the recipient DEFS for a given AA (or set of one or more AAs) being transferred to the recipient DEFS, various optimizations may be employed

to avoid copying the entire content of a given metafile into the message. According to one embodiment, only the portion or part of the metafile corresponding to the storage that is being moved is transferred to the recipient DEFS. For example, as described further below, in accordance with a first optimization, one or more level-0 (L0) PVBNs pointing to respective data blocks containing metafile data for the given AA may be included in the message. In a second optimization, one or more level-1 (L1) PVBNs, which in turn each point to multiple L0 PVBNs, may be included in the message and those of the L0 PVBNs.

[0053] As used herein, a “full allocation area” or “a full AA” generally refers to a partial AA for which a predetermined or configurable threshold of PVBNs of the partial AA are marked as used or in use (containing valid data). For example, a partial AA may be considered to be a full AA when 95% or more of its storage space is in use.

[0054] As used herein, a “storage pod” generally refers to a group of disks containing multiple RAID groups that are accessible from all storage systems (nodes) of a distributed storage system (cluster).

[0055] As used herein, a “data pod” generally refers to a set of storage systems (nodes) that share the same storage pod. In some examples, a data pod refers to a single cluster of nodes representing a distributed storage system. In other examples, there can be multiple data pods in a cluster. Data pods may be used to limit the fault domain and there can be multiple HA pairs of nodes within a data pod.

[0056] As used herein, an “active map” is a metafile or data structure that contains file system metadata information indicative of which PVBNs of a distributed file system are in use. In one embodiment, the active map is represented in the form of a sparse bit map in which each PVBN of a global PVBN space of a storage pod has a corresponding Boolean value (or truth value) represented as a single bit, for example, in which the true (1) indicates the corresponding PVBN is in use and false (0) indicates the corresponding PVBN is not in use.

[0057] As used herein, a “dynamically extensible file system” or a “DEFS” generally refers to a file system of a data pod or a cluster that has visibility into the entire global PVBN space of a storage pod and hosts multiple volumes. A DEFS may be thought of as a data container or a storage container (which may be referred to as a storage segment container) to which AAs are assigned, thereby resulting in a more flexible and enhanced version of a node-level aggregate. As described further herein (for example, in connection with automatic space balancing), the storage space associated with one or more AAs of a given DEFS may be dynamically transferred or moved on demand to any other DEFS in the cluster by changing the ownership of the one or more AAs and moving associated AA tracking data structures as appropriate. This provides the unique ability to independently scale each DEFS of a cluster. For example, DEFSs can shrink or grow dynamically over time to meet their respective storage needs and silos of storage space are avoided. In one embodiment, a distributed file system comprises multiple instances of the WAFL Copy-on-Write file system running on respective storage systems (nodes) of a distributed storage system (cluster) that represents the data pod. In various examples described herein, a given storage system (node) of a distributed storage system (cluster) may own one or more DEFSs including, for example, a log DEFS for hosting an operation log or journal of certain storage

operations that have been performed by the node since the last CP and a data DEFS for hosting customer volumes or logical unit numbers (LUNs). As described further below, the partitioning/division of a storage pod into AAs (creation of a disaggregated storage space) and the distribution of ownership of AAs among DEFSs of multiple nodes of a cluster may facilitate implementation of a distributed storage system having a disaggregated storage architecture. In various examples described herein, each storage system may have its own portion of disaggregated storage to which it has the exclusive ability to perform write access, thereby simplifying storage management by, among other things, not requiring implementation of access control mechanisms, for example, in the form of locks. At the same time, each storage system also has visibility into the entirety of a global PVBN space, thereby allowing read access by a given storage system to any portion of the disaggregated storage regardless of which node of the cluster is the current owner of the underlying allocation areas. Based on disclosure provided herein, those skilled in the art will understand there are at least two types of disaggregation represented/achieved within various examples, including (i) the disaggregation of storage space provided by a storage pod by dividing or partitioning the storage space into AAs the ownership of which can be fluidly changed from one DEFS to another on demand and (ii) the disaggregation of the storage architecture into independent components, including the decoupling of processing resources and storage resources, thereby allowing them to be independently scaled. In one embodiment, the former (which may also be referred to as modular storage, partitioned storage, adaptable storage, or fluid storage) facilitates the latter.

[0058] As used herein, an “allocation area map,” “AA map,” or “AA owner file” generally refers to a per dynamically extensible file system data structure or file (e.g., a metafile) that contains metadata information at an AA-level of granularity indicative of which AAs are assigned to or “owned” by a given dynamically extensible file system.

[0059] A “node-level aggregate” generally refers to a file system of a single storage system (node) that holds multiple volumes created over one or more RAID groups, in which the node owns the entire PVBN space of the collection of disks of the one or more RAID groups. Node-level aggregates are only accessible from a single storage system (node) of a distributed storage system (cluster) at a time.

[0060] As used herein, an “index node” or “inode” generally refers to a file data structure maintained by a file system that stores metadata for data containers (e.g., directories, subdirectories, disk files, etc.). An inode may include, among other things, location, file size, permissions needed to access a given file with which it is associated as well as creation, read, and write timestamps, and one or more flags.

[0061] As used herein, a “storage volume” or “volume” generally refers to a container in which applications, databases, and file systems store data. A volume is a logical component created for the host to access storage on a storage array. A volume may be created from the capacity available in storage pod, a pool, or a volume group. A volume has a defined capacity. Although a volume might consist of more than one drive, a volume appears as one logical component to the host. Non-limiting examples of a volume include a flexible volume and a flexgroup volume.

[0062] As used herein, a “flexible volume” generally refers to a type of storage volume that may be efficiently

distributed across multiple storage devices. A flexible volume may be capable of being resized to meet changing business or application requirements. In some embodiments, a storage system may provide one or more aggregates and one or more storage volumes distributed across a plurality of nodes interconnected as a cluster. Each of the storage volumes may be configured to store data such as files and logical units. As such, in some embodiments, a flexible volume may be comprised within a storage aggregate and further comprises at least one storage device. The storage aggregate may be abstracted over a RAID plex where each plex comprises a RAID group. Moreover, each RAID group may comprise a plurality of storage disks. As such, a flexible volume may comprise data storage spread over multiple storage disks or devices. A flexible volume may be loosely coupled to its containing aggregate. A flexible volume can share its containing aggregate with other flexible volumes. Thus, a single aggregate can be the shared source of all the storage used by all the flexible volumes contained by that aggregate. A non-limiting example of a flexible volume is a NetApp ONTAP Flex Vol volume.

[0063] As used herein, a “flexgroup volume” generally refers to a single namespace that is made up of multiple constituent/member volumes. A non-limiting example of a flexgroup volume is a NetApp ONTAP FlexGroup volume that can be managed by storage administrators, and which acts like a NetApp Flex Vol volume. In the context of a flexgroup volume, “constituent volume” and “member volume” are interchangeable terms that refer to the underlying volumes (e.g., flexible volumes) that make up the flexgroup volume.

Example Distributed Storage System Cluster

[0064] FIG. 1 is a block diagram illustrating a plurality of nodes **110a-b** interconnected as a cluster **100** in accordance with an embodiment of the present disclosure. In the context of the present example, the nodes **110a-b** comprise various functional components that cooperate to provide a distributed storage system architecture of the cluster **100**. To that end, in the context of the present example, each node is generally organized as a network element (e.g., network element **120a** or **120b**) and a disk element (e.g., disk element **150a** or **150b**). The network element includes functionality that enables the node to connect to clients (e.g., client **180**) over a computer network **140**, while each disk element **350** connects to one or more storage devices, such as disks, of one or more disk arrays (not shown) or of one or more storage shelves (not shown), represented as a single shared storage pod **145**.

[0065] In the context of the present example, the nodes **110a-b** are interconnected by a cluster switching fabric **151** which, in an example, may be embodied as a Gigabit Ethernet switch. It should be noted that while there is shown an equal number of network and disk elements in the illustrative cluster **100**, there may be differing numbers of network and/or disk elements. For example, there may be a plurality of network elements and/or disk elements interconnected in a cluster configuration **100** that does not reflect a one-to-one correspondence between the network and disk elements. As such, the description of a node comprising one network element and one disk element should be taken as illustrative only.

[0066] Clients may be general-purpose computers configured to interact with the node in accordance with a client/

server model of information delivery. That is, each client (e.g., client **180**) may request the services of the node, and the node may return the results of the services requested by the client, by exchanging packets over the network **140**. The client may issue packets including file-based access protocols, such as the Common Internet File System (CIFS) protocol or Network File System (NFS) protocol, over the Transmission Control Protocol/Internet Protocol (TCP/IP) when accessing information in the form of files and directories. Alternatively, the client may issue packets including block-based access protocols, such as the Small Computer Systems Interface (SCSI) protocol encapsulated over TCP (iSCSI) and SCSI encapsulated over Fibre Channel (FCP), when accessing information in the form of blocks. In various examples described herein, an administrative user (not shown) of the client may make use of a user interface (UI) presented by the cluster or a command line interface (CLI) of the cluster to, among other things, establish a data protection relationship between a source volume and a destination volume (e.g., a mirroring relationship specifying one or more policies associated with creation, retention, and transfer of snapshots), defining snapshot and/or backup policies, and association of snapshot policies with snapshots.

[0067] Disk elements **150a** and **150b** are illustratively connected to disks (not shown) within that may be organized into disk arrays within the storage pod **145**. Alternatively, storage devices other than disks may be utilized, e.g., flash memory, optical storage, solid state devices, etc. As such, the description of disks should be taken as exemplary only.

[0068] In general, various embodiments envision a cluster (e.g., cluster **100**) in which every node (e.g., nodes **110a-b**) can essentially talk to every storage device (e.g., disk) in the storage pod **145**. This is in contrast to the distributed storage system architecture described with reference to FIG. 5. In examples described herein, all nodes (e.g., nodes **110a-b**) of the cluster have visibility and read access to an entirety of a global PVBN space of the storage pod **145**, for example, via an interconnect layer **142**. As described further below, according to one embodiment, the storage within the storage pod **145** is grouped into distinct allocation areas (AAs) than can be assigned to a given dynamically extensible file system (DEFS) of a node to facilitate implementation disaggregated storage. In examples described herein, the AAs assigned to a given DEFS may be said to “own” the assigned AAs and the node owning the given DEFS has the exclusive write access to the associated PVBNs and the exclusive ability to perform write allocation from such blocks. In one embodiment, each node has its own view of a portion of the disaggregated storage represented by the assignment of, for example, via respective allocation area (AA) maps and active maps. This granular assignment of AAs and ability to fluidly change ownership of AAs as needed facilitates the elimination of per-node storage silos and provides higher and more predictable performance, which further translate into improved storage utilization and improvements in cost effectiveness of the storage solution.

[0069] Depending on the particular implementation, the interconnect layer **142** may be represented by an intermediate switching topology or some other interconnectivity layer or disk switching layer between the disks in the storage pod **145** and the nodes. Non-limiting examples of the interconnect layer **150** include one or more fiber channel switches or one or more non-volatile memory express

(NVMe) fabric switches. Additional details regarding the storage pod **145**, DEFSs, AA maps, active maps, and the use, ownership, and sharing (transferring of ownership) of AAs are described further below.

Example Storage System Node

[0070] FIG. 2 is a block diagram of a node **200** that is illustratively embodied as a storage system comprising a plurality of processors (e.g., processors **222a-b**), a memory **224**, a network adapter **225**, a cluster access adapter **226**, a storage adapter **228** and local storage **230** interconnected by a system bus **223**. Node **200** may be analogous to nodes **110a** and **110b** of FIG. 1. The local storage **230** comprises one or more storage devices, such as disks, utilized by the node to locally store configuration information (e.g., in configuration table **235**). The cluster access adapter **226** comprises a plurality of ports adapted to couple the node **200** to other nodes of the cluster (e.g., cluster **100**). Illustratively, Ethernet is used as the clustering protocol and interconnect media, although it will be apparent to those skilled in the art that other types of protocols and interconnects may be utilized within the cluster architecture described herein. Alternatively, where the network elements and disk elements are implemented on separate storage systems or computers, the cluster access adapter **226** is utilized by the network and disk element for communicating with other network and disk elements in the cluster.

[0071] In the context of the present example, each node **200** is illustratively embodied as a dual processor storage system executing a storage operating system **210** that implements a high-level module, such as a file system, to logically organize the information as a hierarchical structure of named directories, files and special types of files called virtual disks (hereinafter generally “blocks”) on the disks. However, it will be apparent to those of ordinary skill in the art that the node **200** may alternatively comprise a single or more than two processor system. Illustratively, one processor (e.g., processor **222a**) may execute the functions of the network element (e.g., network element **120a** or **120b**) on the node, while the other processor (e.g., processor **222b**) may execute the functions of the disk element (e.g., disk element **150a** or **150b**).

[0072] The memory **224** illustratively comprises storage locations that are addressable by the processors and adapters for storing software program code and data structures associated with the subject matter of the disclosure. The processor and adapters may, in turn, comprise processing elements and/or logic circuitry configured to execute the software code and manipulate the data structures. The storage operating system **210**, portions of which is typically resident in memory and executed by the processing elements, functionally organizes the node **200** by, inter alia, invoking storage operations in support of the storage service implemented by the node. It will be apparent to those skilled in the art that other processing and memory means, including various computer readable media, may be used for storing and executing program instructions pertaining to the disclosure described herein.

[0073] The network adapter **225** comprises a plurality of ports adapted to couple the node **200** to one or more clients (e.g., client **180**) over point-to-point links, wide area networks, virtual private networks implemented over a public network (Internet) or a shared local area network. The network adapter **225** thus may comprise the mechanical,

electrical and signaling circuitry needed to connect the node to a network (e.g., computer network **140**). Illustratively, the network may be embodied as an Ethernet network or a Fibre Channel (FC) network. Each client (e.g., client **180**) may communicate with the node over network by exchanging discrete frames or packets of data according to pre-defined protocols, such as TCP/IP.

[0074] The storage adapter **228** cooperates with the storage operating system **210** executing on the node **200** to access information requested by the clients. The information may be stored on any type of attached array of writable storage device media such as video tape, optical, DVD, magnetic tape, bubble memory, electronic random access memory, micro-electromechanical and any other similar media adapted to store information, including data and parity information. However, as illustratively described herein, the information is stored on disks (e.g., associated with storage pod **145**). The storage adapter comprises a plurality of ports having input/output (I/O) interface circuitry that couples to the disks over an I/O interconnect arrangement, such as a conventional high-performance, FC link topology.

[0075] Storage of information on each disk array may be implemented as one or more storage “volumes” that comprise a collection of physical storage disks or cloud volumes cooperating to define an overall logical arrangement of volume block number (VBN) space on the volume(s). Each logical volume is generally, although not necessarily, associated with its own file system. The disks within a logical volume/file system are typically organized as one or more groups, wherein each group may be operated as a Redundant Array of Independent (or Inexpensive) Disks (RAID). Most RAID implementations, such as a RAID-4 level implementation, enhance the reliability/integrity of data storage through the redundant writing of data “stripes” across a given number of physical disks in the RAID group, and the appropriate storing of parity information with respect to the striped data. An illustrative example of a RAID implementation is a RAID-4 level implementation, although it should be understood that other types and levels of RAID implementations may be used in accordance with the inventive principles described herein.

[0076] While in the context of the present example, the node may be a physical host, it is to be appreciated the node may be implemented in virtual form. For example, a storage system may be run (e.g., on a VM or as a containerized instance, as the case may be) within a public cloud provider. As such, a cluster representing a distributed storage system may be comprised of multiple physical nodes (e.g., node **200**) or multiple virtual nodes (virtual storage systems).

Example Storage Operating System

[0077] To facilitate access to the disks (e.g., disks within one or more disk arrays of a storage pod, such as storage pod **145** of FIG. 1), a storage operating system (e.g., storage operating system **300**, which may be analogous to storage operating system **210**) may implement a write-anywhere file system that cooperates with one or more virtualization modules to “virtualize” the storage space provided by disks. The file system logically organizes the information as a hierarchical structure of named directories and files on the disks. Each “on-disk” file may be implemented as set of disk blocks configured to store information, such as data, whereas the directory may be implemented as a specially

formatted file in which names and links to other files and directories are stored. The virtualization module(s) allow the file system to further logically organize information as a hierarchical structure of blocks on the disks that are exported as named logical unit numbers (LUNs).

[0078] Illustratively, the storage operating system may be the Data ONTAP operating system available from NetApp, Inc., San Jose, Calif. that implements the WAFL file system. However, it is expressly contemplated that any appropriate storage operating system may be enhanced for use in accordance with the inventive principles described herein. As such, where the term “WAFL” is employed, it should be taken broadly to refer to any file system that is otherwise adaptable to the teachings of this disclosure.

[0079] FIG. 3 is a block diagram illustrating a storage operating system 300 in accordance with an embodiment of the present disclosure. In the context of the present example, the storage operating system 300 is shown including a series of software layers organized to form an integrated network protocol stack or, more generally, a multi-protocol engine 325 that provides data paths for clients to access information stored on the node using block and file access protocols. The multi-protocol engine includes a media access layer 312 of network drivers (e.g., gigabit Ethernet drivers) that interfaces to network protocol layers, such as the IP layer 314 and its supporting transport mechanisms, the TCP layer 316 and the User Datagram Protocol (UDP) layer 315. A file system protocol layer provides multi-protocol file access and, to that end, includes support for the Direct Access File System (DAFS) protocol 318, the NFS protocol 320, the CIFS protocol 322 and the Hypertext Transfer Protocol (HTTP) protocol 324. A VI layer 326 implements the VI architecture to provide direct access transport (DAT) capabilities, such as ROMA, as required by the DAFS protocol 318. An iSCSI driver layer 328 provides block protocol access over the TCP/IP network protocol layers, while a FC driver layer 330 receives and transmits block access requests and responses to and from the node. The FC and iSCSI drivers provide FC-specific and iSCSI-specific access control to the blocks and, thus, manage exports of LUNs to either iSCSI or FCP or, alternatively, to both iSCSI and FCP when accessing the blocks on the node (e.g., node 200).

[0080] In addition, the storage operating system may include a series of software layers organized to form a storage server 365 that provides data paths for accessing information stored on the disks (e.g., disks 130) of the node. To that end, the storage server 365 includes a file system module 360 in cooperating relation with a remote access module 370, a RAID system module 380 and a disk driver system module 390. The RAID system 380 manages the storage and retrieval of information to and from the volumes/disks in accordance with I/O operations, while the disk driver system 390 implements a disk access protocol such as, e.g., the SCSI protocol.

[0081] The file system 360 may implement a virtualization system of the storage operating system 300 through the interaction with one or more virtualization modules illustratively embodied as, for example, a virtual disk (vdisk) module (not shown) and a SCSI target module 335. The SCSI target module 335 is generally disposed between the FC and iSCSI drivers 328, 330 and the file system 360 to provide a translation layer of the virtualization system between the block (LUN) space and the file system space, where LUNs are represented as blocks.

[0082] The file system 360 is illustratively a message-based system that provides logical volume management capabilities for use in access to the information stored on the storage devices, such as disks. That is, in addition to providing file system semantics, the file system 360 provides functions normally associated with a volume manager. These functions include (i) aggregation of the disks, (ii) aggregation of storage bandwidth of the disks, and (iii) reliability guarantees, such as mirroring and/or parity (RAID). The file system 360 illustratively implements an exemplary a file system having an on-disk format representation that is block-based using, e.g., 4 kilobyte (KB) blocks and using index nodes (“inodes”) to identify files and file attributes (such as creation time, access permissions, size and block location). The file system uses files to store metadata describing the layout of its file system; these metadata files include, among others, an inode file. A file handle, i.e., an identifier that includes an inode number, is used to retrieve an inode from disk.

[0083] Broadly stated, all inodes of the write-anywhere file system are organized into the inode file. A file system (fs) info block specifies the layout of information in the file system and includes an inode of a file that includes all other inodes of the file system. Each logical volume (file system) has an fsinfo block that is preferably stored at a fixed location within, e.g., a RAID group. The inode of the inode file may directly reference (point to) data blocks of the inode file or may reference indirect blocks of the inode file that, in turn, reference data blocks of the inode file. Within each data block of the inode file are embedded inodes, each of which may reference indirect blocks that, in turn, reference data blocks of a file.

[0084] Operationally, a request from a client (e.g., client 180) is forwarded as a packet over a computer network (e.g., computer network 140) and onto a node (e.g., node 200) where it is received at a network adaptor (e.g., network adaptor 225). A network driver (of layer 312 or layer 330) processes the packet and, if appropriate, passes it on to a network protocol and file access layer for additional processing prior to forwarding to the write-anywhere file system 360. Here, the file system generates operations to load (retrieve) the requested data from disk 130 if it is not resident “in core”, i.e., in memory 224. If the information is not in memory, the file system 360 indexes into the inode file using the inode number to access an appropriate entry and retrieve a logical VBN. The file system then passes a message structure including the logical VBN to the RAID system 380; the logical VBN is mapped to a disk identifier and disk block number (disk,dbn) and sent to an appropriate driver (e.g., SCSI) of the disk driver system 390. The disk driver accesses the dbn from the specified disk 130 and loads the requested data block(s) in memory for processing by the node. Upon completion of the request, the node (and operating system) returns a reply to the client 180 over the network 140.

[0085] The remote access module 370 is operatively interfaced between the file system module 360 and the RAID system module 380. Remote access module 370 is illustratively configured as part of the file system to implement the functionality to determine whether a newly created data container, such as a subdirectory, should be stored locally or remotely. Alternatively, the remote access module 370 may be separate from the file system. As such, the description of the remote access module being part of the file system

should be taken as exemplary only. Further, the remote access module 370 determines which remote flexible volume should store a new subdirectory if a determination is made that the subdirectory is to be stored remotely. More generally, the remote access module 370 implements the heuristics algorithms used for the adaptive data placement. However, it should be noted that the use of a remote access module should be taken as illustrative. In alternative aspects, the functionality may be integrated into the file system or other module of the storage operating system. As such, the description of the remote access module 370 performing certain functions should be taken as exemplary only.

[0086] It should be noted that the software “path” through the storage operating system layers described above needed to perform data storage access for the client request received at the node may alternatively be implemented in hardware. That is, a storage access request data path may be implemented as logic circuitry embodied within a field programmable gate array (FPGA) or an application specific integrated circuit (ASIC). This type of hardware implementation increases the performance of the storage service provided by node 200 in response to a request issued by client 180. Alternatively, the processing elements of adapters 225, 228 may be configured to offload some or all of the packet processing and storage access operations, respectively, from processor 222, to thereby increase the performance of the storage service provided by the node. It is expressly contemplated that the various processes, architectures and procedures described herein can be implemented in hardware, firmware or software.

[0087] As used herein, the term “storage operating system” generally refers to the computer-executable code operable on a computer to perform a storage function that manages data access and may, in the case of a node (e.g., node 200), implement data access semantics of a general purpose operating system. The storage operating system can also be implemented as a microkernel, an application program operating over a general-purpose operating system, such as UNIX or Windows NT, or as a general-purpose operating system with configurable functionality, which is configured for storage applications as described herein.

[0088] In addition, it will be understood to those skilled in the art that aspects of the disclosure described herein may apply to any type of special-purpose (e.g., file server, filer or storage serving appliance) or general-purpose computer, including a standalone computer or portion thereof, embodied as or including a storage system. Moreover, the teachings contained herein can be adapted to a variety of storage system architectures including, but not limited to, a network-attached storage environment, a storage area network and disk assembly directly attached to a client or host computer. The term “storage system” should therefore be taken broadly to include such arrangements in addition to any subsystems configured to perform a storage function and associated with other equipment or systems. It should be noted that while this description is written in terms of a write anywhere file system, the teachings of the subject matter may be utilized with any suitable file system, including a write in place file system.

Example Cluster Fabric (CF) Protocol

[0089] Illustratively, the storage server 365 is embodied as disk element (or disk blade 350, which may be analogous to disk element 150a or 150b) of the storage operating system

300 to service one or more volumes of array 160. In addition, the multi-protocol engine 325 is embodied as network element (or network blade 310, which may be analogous to network element 120a or 120b) to (i) perform protocol termination with respect to a client issuing incoming data access request packets over the network (e.g., network 140), as well as (ii) redirect those data access requests to any storage server 365 of the cluster (e.g., cluster 100). Moreover, the network element 310 and disk element 350 cooperate to provide a highly scalable, distributed storage system architecture of the cluster. To that end, each module may include a cluster fabric (CF) interface module (e.g., CF interface 340a and 340b) adapted to implement intra-cluster communication among the nodes (e.g., node 110a and 110b). In the context of a distributed storage architecture as described below with reference to FIG. 5 in which node-level aggregates are employed, the CF protocol facilitates, among other things, internode communications relating to data access requests. It is to be appreciated such internode communications relating to data access requests are not needed in the context of a distributed storage architecture as described below with reference to FIG. 6A in which each node of a cluster has visibility and access to the entirety of a global PVBN space of a storage pod (via their respective DEFSs). However, in various embodiments, some limited amount of internode communications, for example, relating to storage space reporting (or simply space reporting) and storage space requests (e.g., requests for donations of AAs) continue to be useful. As described further below, such internode communications may make use of the CF protocol or other forms of internode communications, including message passing via on-wire communications and/or the use of one or more persistent message queues (or on-disk message queues), which may make use of the fact that all nodes can read from all disk of a storage pod. For example, a persistent message queue may be maintained at the node and/or DEFS-level of granularity in which each node and/or DEFS has a message queue to which others can post messages destined for the node or DEFS (as the case may be). In one embodiment, each DEFS has an associated inbound queue on which it receives messages sent by another DEFS in the cluster and an associated outbound queue on which it posts messages intended for delivery to another DEFS in the cluster.

[0090] The protocol layers, e.g., the NFS/CIFS layers and the iSCSI/FC layers, of the network element 310 may function as protocol servers that translate file-based and block based data access requests from clients into CF protocol messages used for communication with the disk element 350. That is, the network element servers may convert the incoming data access requests into file system primitive operations (commands) that are embedded within CF messages by the CF interface module 340 for transmission to the disk elements of the cluster.

[0091] Further, in an illustrative aspect of the disclosure, the network element and disk element are implemented as separately scheduled processes of storage operating system 300; however, in an alternate aspect, the modules may be implemented as pieces of code within a single operating system process. Communication between a network element and disk element may thus illustratively be effected through the use of message passing between the modules although, in the case of remote communication between a network element and disk element of different nodes, such message

passing occurs over a cluster switching fabric (e.g., cluster switching fabric **151**). A known message-passing mechanism provided by the storage operating system to transfer information between modules (processes) is the Inter Process Communication (IPC) mechanism. The protocol used with the IPC mechanism is illustratively a generic file and/or block-based “agnostic” CF protocol that comprises a collection of methods/functions constituting a CF application programming interface (API). Examples of such an agnostic protocol are the SpinFS and SpinNP protocols available from NetApp, Inc.

[0092] The CF interface module **340** implements the CF protocol for communicating file system commands among the nodes or modules of cluster. Communication may be illustratively effected by the disk element exposing the CF API to which a network element (or another disk element) issues calls. To that end, the CF interface module **340** may be organized as a CF encoder and CF decoder. The CF encoder of, e.g., CF interface **340a** on network element **310** encapsulates a CF message as (i) a local procedure call (LPC) when communicating a file system command to a disk element **350** residing on the same node **200** or (ii) a remote procedure call (RPC) when communicating the command to a disk element residing on a remote node of the cluster **100**. In either case, the CF decoder of CF interface **340b** on disk element **350** de-encapsulates the CF message and processes the file system command.

[0093] Illustratively, the remote access module **370** may utilize CF messages to communicate with remote nodes to collect information relating to remote flexible volumes. A CF message is used for RPC communication over the switching fabric between remote modules of the cluster; however, it should be understood that the term “CF message” may be used generally to refer to LPC and RPC communication between modules of the cluster. The CF message includes a media access layer, an IP layer, a UDP layer, a reliable connection (RC) layer and a CF protocol layer. The CF protocol is a generic file system protocol that may convey file system commands related to operations contained within client requests to access data containers stored on the cluster; the CF protocol layer is that portion of a message that carries the file system commands. Illustratively, the CF protocol is datagram based and, as such, involves transmission of messages or “envelopes” in a reliable manner from a source (e.g., a network element **310**) to a destination (e.g., a disk element **350**). The RC layer implements a reliable transport protocol that is adapted to process such envelopes in accordance with a connectionless protocol, such as UDP.

Example File System Layout

[0094] In one embodiment, a data container is represented in the write-anywhere file system as an inode data structure adapted for storage on the disks of a storage pod (e.g., storage pod **145**). In such an embodiment, an inode includes a metadata section and a data section. The information stored in the metadata section of each inode describes the data container (e.g., a file, a snapshot, etc.) and, as such, includes the type (e.g., regular, directory, vdisk) of file, its size, time stamps (e.g., access and/or modification time) and ownership (e.g., user identifier (UID) and group ID (GID), of the file, and a generation number. The contents of the data section of each inode may be interpreted differently depending upon the type of file (inode) defined within the type field.

For example, the data section of a directory inode includes metadata controlled by the file system, whereas the data section of a regular inode includes file system data. In this latter case, the data section includes a representation of the data associated with the file.

[0095] Specifically, the data section of a regular on-disk inode may include file system data or pointers, the latter referencing 4 KB data blocks on disk used to store the file system data. Each pointer is preferably a logical VBN to facilitate efficiency among the file system and the RAID system when accessing the data on disks. Given the restricted size (e.g., 128 bytes) of the inode, file system data having a size that is less than or equal to 64 bytes is represented, in its entirety, within the data section of that inode. However, if the length of the contents of the data container exceeds 64 bytes but less than or equal to 64 KB, then the data section of the inode (e.g., a first level inode) comprises up to 16 pointers, each of which references a 4 KB block of data on the disk.

[0096] Moreover, if the size of the data is greater than 64 KB but less than or equal to 64 megabytes (MB), then each pointer in the data section of the inode (e.g., a second level inode) references an indirect block (e.g., a first level L1 block) that contains 224 pointers, each of which references a 4 KB data block on disk. For file system data having a size greater than 64 MB, each pointer in the data section of the inode (e.g., a third level L3 inode) references a double-indirect block (e.g., a second level L2 block) that contains 224 pointers, each referencing an indirect (e.g., a first level L1) block. The indirect block, in turn, which contains 224 pointers, each of which references a 4 kB data block on disk. When accessing a file, each block of the file may be loaded from disk into memory (e.g., memory **224**). In other embodiments, higher levels are also possible that may be used to handle larger data container sizes.

[0097] When an on-disk inode (or block) is loaded from disk into memory, its corresponding in-core structure embeds the on-disk structure. The in-core structure is a block of memory that stores the on-disk structure plus additional information needed to manage data in the memory (but not on disk). The additional information may include, e.g., a “dirty” bit. After data in the inode (or block) is updated/modified as instructed by, e.g., a write operation, the modified data is marked “dirty” using the dirty bit so that the inode (block) can be subsequently “flushed” (stored) to disk.

[0098] According to one embodiment, a file in a file system comprises a buffer tree that provides an internal representation of blocks for a file loaded into memory and maintained by the write-anywhere file system **360**. A root (top-level) buffer, such as the data section embedded in an inode, references indirect (e.g., level 1) blocks. In other embodiments, there may be additional levels of indirect blocks (e.g., level 2, level 3) depending upon the size of the file. The indirect blocks (e.g., and inode) includes pointers that ultimately reference data blocks used to store the actual data of the file. That is, the data of file are contained in data blocks and the locations of these blocks are stored in the indirect blocks of the file. Each level 1 indirect block may include pointers to as many as 224 data blocks. According to the “write anywhere” nature of the file system, these blocks may be located anywhere on the disks.

[0099] In one embodiment, a file system layout is provided that apportions an underlying physical volume into

one or more virtual volumes (or flexible volumes) of a storage system, such as node **200**. In such an embodiment, the underlying physical volume is an aggregate comprising one or more groups of disks, such as RAID groups, of the node. The aggregate has its own physical volume block number (PVBN) space and maintains metadata, such as block allocation structures, within that PVBN space. Each flexible volume has its own virtual volume block number (VVBN) space and maintains metadata, such as block allocation structures, within that VVBN space. Each flexible volume is a file system that is associated with a container file; the container file is a file in the aggregate that contains all blocks used by the flexible volume. Moreover, each flexible volume comprises data blocks and indirect blocks that contain block pointers that point at either other indirect blocks or data blocks.

[0100] In a further embodiment, PVBNs are used as block pointers within buffer trees of files stored in a flexible volume. This “hybrid” flexible volume example involves the insertion of only the PVBN in the parent indirect block (e.g., inode or indirect block). On a read path of a logical volume, a “logical” volume (vol) info block has one or more pointers that reference one or more fsinfo blocks, each of which, in turn, points to an inode file and its corresponding inode buffer tree. The read path on a flexible volume is generally the same, following PVBNs (instead of VVBNs) to find appropriate locations of blocks; in this context, the read path (and corresponding read performance) of a flexible volume is substantially similar to that of a physical volume. Translation from PVBN-to-disk.dbn occurs at the file system/RAID system boundary of the storage operating system **300**.

[0101] In a dual VBN hybrid flexible volume example, both a PVBN and its corresponding VVBN are inserted in the parent indirect blocks in the buffer tree of a file. That is, the PVBN and VVBN are stored as a pair for each block pointer in most buffer tree structures that have pointers to other blocks, e.g., level 1 (L1) indirect blocks, inode file level 0 (L0) blocks.

[0102] A root (top-level) buffer, such as the data section embedded in an inode, references indirect (e.g., level 1) blocks. Note that there may be additional levels of indirect blocks (e.g., level 2, level 3) depending upon the size of the file. The indirect blocks (and inode) include PVBN/VVBN pointer pair structures that ultimately reference data blocks used to store the actual data of the file. The PVBNs reference locations on disks of the aggregate, whereas the VVBNs reference locations within files of the flexible volume. The use of PVBNs as block pointers in the indirect blocks provides efficiencies in the read paths, while the use of VVBN block pointers provides efficient access to required metadata. That is, when freeing a block of a file, the parent indirect block in the file contains readily available VVBN block pointers, which avoids the latency associated with accessing an owner map to perform PVBN-to-VVBN translations; yet, on the read path, the PVBN is available.

Example Hierarchical Inode Tree

[0103] FIG. 4 is a block diagram illustrating a tree of blocks **400** representing a simplified view of an example a file system layout in accordance with an embodiment of the present disclosure. In one embodiment, the data storage system nodes (e.g., data storage systems **110a-b**) make use of a write anywhere file system (e.g., the WAFL file system). The write anywhere file system may represent a UNIX

compatible file system that is optimized for network file access. In the context of the present example, the write anywhere file system is a block-based file system that represents file system data (e.g., a block map file and an inode map file), metadata files, and data containers (e.g., volumes, subdirectories, and regular files) in a tree of blocks (e.g., tree of blocks **400**), which may also be referred to herein as a buffer tree. Keeping metadata in files (which may be referred to herein as metafiles) allows the file system to write metadata blocks anywhere on disk and makes it easier to increase the size of the file system on the fly.

[0104] In this simplified example, the tree of blocks **400** has a root inode **410**, which describes an inode map file (not shown), made up of inode file indirect blocks **420** and inode file data blocks **430**. In this example, the file system uses inodes (e.g., inode file data blocks **430**) to describe data containers representing files (e.g., file **431a** and file **431b**). In one embodiment, each inode contains a predetermined number of block pointers (e.g., 16 block pointers) to indicate which blocks (e.g., of 4 KB) belong to a given data container (e.g., a file). Inodes for data containers smaller than 64 KB may use its block pointers to point to or otherwise identify the location of file data blocks or simply data blocks (e.g., regular file data blocks, which may also be referred to herein as L0 blocks **450**). In this example, a given L0 block of L0 blocks **450** corresponds to a block of data on a particular disk. Inodes for files greater than 64 MB may point to indirect blocks (e.g., regular file indirect blocks, which may also be referred to herein as L1 blocks **440**), which contain pointers to or locations of actual file data on disk. Inodes for larger files or data containers may point to doubly indirect blocks. For very small files, data may be stored in the inode itself in place of the block pointers.

[0105] As will be appreciated by those skilled in the art given the above-described file system layout, yet another advantage of DEFSs are their ability to facilitate storage space balancing and/or load balancing. This comes from the fact that the entire global PVBN space of a storage pod is visible to all DEFSs of the cluster and therefore any given DEFS can get access to an entire file by copying the top-most PVBN from the inode to or from another tree.

[0106] Exemplary optimizations for efficiently transferring metafile data for one or more AAs for which the ownership is being transferred from one DEFS (e.g., a donor DEFS donating the one or more AAs) to another DEFS (e.g., a recipient DEFS receiving the one or more AAs) are described below with reference to FIGS. **8-10B**.

Example of a Distributed Storage System Architecture with Storage Silos

[0107] FIG. 5 is a block diagram illustrating a distributed storage system architecture **500** in which the entirety of a given disk and a given RAID group are owned by an aggregate and the aggregate file system is only visible from one node, thereby resulting in silos of storage space. In the context of FIG. 5, node **510a** and node **510b** may represent a two-node cluster in which the nodes are high-availability (HA) partners. For example, one node may represent a primary node and the other may represent a secondary node in which pairwise disk connectively supports a pairwise failover model. As shown, each node includes respective active maps (e.g., active map **541a** and active map **541b**) and a sets of disks (in this case, ten disks) they can talk to. The nodes may partition the disks among themselves as aggre-

gates (e.g., data aggregate **520a** and data aggregate **520b**) and at steady state both nodes will work on their own subset of disks representing a one or more RAID groups (in this case, four data disks and one parity disk, forming a single RAID group). A RAID layer or subsystem (not shown) of a storage operating system (not shown) of each node may present respective separate and independent PVBN spaces (e.g., PVBN space **540a** and PVBN space **540b**) to a file system layer (not shown) of the node.

[0108] In this example, therefore, data aggregate **520a** has visibility only to a first PVBN space (e.g., PVBN space **540a**) and data aggregate **520b** has visibility only to a second PVBN space (e.g., PVBN space **540b**). When data is stored to volume **530a** or **530b**, it is striped across the subset of disks that are part of data aggregate **520a**; and when data is stored to volume **530c** or **530d**, it is striped across the subset of disks that are part of data aggregate **520b**. Active map **541a** is a data structure (e.g., a bit map with one bit per PVBN) that identifies the PVBNs within PVBN space **540a** that are in use by data aggregate **520a**. Similarly, active map **541b** is a data structure (e.g., a bit map with one bit per PVBN) that identifies the PVBNs within PVBN space **540b** that are in use by data aggregate **520b**.

[0109] As can be seen, for any given disk, the entire disk is owned by a particular aggregate and the aggregate file system is only visible from one node. Similarly, for any given RAID group, the available storage space of the entire RAID group is useable only by a single node. There are various other disadvantages to the architecture shown in FIG. 5. For example, moving a volume from one aggregate to another requires copying of data (e.g., reading all the blocks used by the volume and writing them to the new location), with an elaborate handover sequence between the aggregates involved. Additionally, there are scenarios in which one data aggregate may run out of storage space while the other still has plentiful free storage space, resulting in ineffective usage of the storage space provided by the disks. While the size of the PVBN space of an aggregate may be increased, doing so typically requires an administrative user to monitor the storage space on each node-level aggregate and add one or more disks and/or RAID groups to the aggregate. As described further below with reference to FIG. 6A, with DEFSs storage space is added to a common pool of storage referred to herein as a “storage pod” and space is available for consumption by any DEFS in the cluster, thereby making space management much simpler and facilitating the automatic balancing of storage space without administrator involvement.

Example Distributed System Architecture Providing Disaggregated Storage

[0110] Before getting into the details of a particular example, various properties, constructs, and principles relating to the use and implementation of DEFSs will now be discussed. As noted above, it is desirable to make the global PVBN space of the entire storage pool available on each DEFS of a data pod, which may include one or more clusters. This feature facilitates the performance of, among other things, instant copy-free moves of volumes from one DEFS to another, for example, in connection with performing load balancing. Creating clones on remote nodes for load balancing is yet another benefit. With a global PVBN space,

support for global data deduplication can also be supported rather than deduplication being limited to node-level aggregates.

[0111] It is also beneficial, in terms of performance, to avoid the use of access control mechanism, such as locks, to coordinate write accesses and write allocation among nodes generally and DEFSs specifically. Such access control mechanisms may be eliminated by specifying, at a per-DEFS level, those portions of the disaggregated storage of the storage pod to which a given DEFS has exclusive write access. For example, as described further below, a DEFS may be limited to use of only the AAs associated with (assigned to or owned by) the DEFS for performing write allocation and write accesses during a CP. Advantageously, given the visibility into the entire global PVBN space, reads can be performed by any DEFS of the cluster from all the PVBNs in the storage pod.

[0112] Each DEFS of a given cluster (or data pod, as the case may be) may start at its own super block. As shown and described with reference to FIG. 6A, a predefined AA (e.g., the first AA) in storage pod may be dedicated for super blocks. In one embodiment, a set of RAID stripes within the predefined super block AA (e.g., the first AA of the storage pod) may be dedicated for super blocks. In this predefined super block AA, ownership may be specified at the granularity of a single RAID stripe instead of at the AA granularity of multiple RAID stripes representing one or more GB (e.g., between approximately 1 GB and 10 GB) of storage space. The location of a super block of a given DEFS can be mathematically derived using an identifier (a DEFS ID) associated with the given DEFS. Since the RAID stripe is already reserved for a super block, it can be replicated on N disks.

[0113] Each DEFS has AAs associated with it, which may be thought of conceptually as the DEFS owning those AAs. In one embodiment, AAs may be tracked within an AA map and persisted within the DEFS filesystem. An AA map may include the DEFS ID in an AA index. While AA ownership information regarding other DEFSs in the cluster may be cached in the AA map of a given DEFS, which may be useful during the PVBN free path, for example, to facilitate freeing of PVBNs of an AA not owned by the given DEFS (which may arise in situations in which partial AAs are donated from one DEFS to another), the authoritative source information regarding the AAs owned by a given DEFS may be presumed to be in the AA map of the given DEFS.

[0114] In support of avoiding storage silos and supporting the more fluid use of disk space across all nodes of a cluster, DEFSs may be allowed to donate partially or completely free AAs to other DEFSs.

[0115] Each DEFS may have its own label information kept in the file system. The label information may be kept in the super block or another well-known location outside of the file system.

[0116] In various examples, there can be multiple DEFSs on a RAID tree. That is, there may be a many-to-one association between DEFSs and a RAID tree, in which each DEFS may have a reference on the RAID tree. The RAID tree can still have multiple RAID groups. In various examples described herein, it is assumed the PVBN space provided by the RAID tree is continuous.

[0117] It may be helpful to have a root DEFS and a data DEFS that are transparent to other subsystems. These DEFSs may be useful for storing information that might be

needed before the file system is brought online. Examples of such information may include controller (node) failover (CFO) and storage failover (SFO) properties/policies. HA is one example of where it might be helpful to bring up a controller (node) failover root DEFS first before giving back the storage failover data DEFSs. HA coordination of bringing down a given DEFS on takeover/giveback may be handled by the file system (e.g., WAFL) since the RAID tree would be up until the node is shutdown.

[0118] DEFS data structures (e.g., DEFS bit maps at the PVBN level, such as active maps and reference count (refcount) maps) may be sparse. That is, they may represent the entire global PVBN space, but only include valid truth values for PVBNs of AAs that are owned by the particular DEFS with which they are associated. When validation of these bit maps is performed by or on behalf of a particular DEFS, the bits should be validated only for the AA areas owned by the particular DEFS. When using such sparse data structures, to get the complete picture of the PVBN space, the data structures in all of the nodes should be taken into consideration. While various DEFS data structures may be discussed herein as if they were separate metafiles, it is to be appreciated, given the visibility by each node into the entire global PVBN space, one or more of such DEFS data structures may be represented as cluster-wide metafiles. Such a cluster-wide metafile may be persisted in a private inode space that is not accessible to end users and the relevant portions for a particular DEFS may be located based on the DEFS ID of the particular DEFS, for example, which may be associated with the appropriate inode (e.g., an L0 block). Similarly, the entirety of such a cluster-wide metafile may be accessible based on a cluster ID, for example, which may be associated with a higher-level inode in the hierarchy (e.g., an L1 block). In any event, each node should generally have all the information it needs to work independently until and unless it runs out of storage space or meets a predetermined or configurable threshold of a storage space metric (e.g., a free space metric or a used space metric), for example, relative to the other nodes of the cluster. At that point, as described further below, as part of a space monitoring and/or a space balancing process, the node may request a portion of AAs of DEFSs owned by one or more of such other nodes be donated so as to increase the useable storage space of one or more DEFSs of the node at issue.

[0119] FIG. 6A is a block diagram illustrating a distributed storage system architecture **600** that provides disaggregated storage in accordance with an embodiment of the present disclosure. Various architectural advantages of the proposed distributed storage system architecture and mechanisms for providing and making use of disaggregated storage include, but are not limited to, the ability to perform automatic space balancing among DEFSs, perform elastic node growth and shrinkage for a cluster, perform elastic storage growth of the storage pod, perform zero-copy file and volume move (migration), perform distributed RAID rebuild, achieve HA cost reduction using volume rehosting, create remote clones, and perform global data deduplication.

[0120] In the context of the present example, the nodes (e.g., node **610a** and **610b**) of a cluster, which may represent a data pod or include multiple data pods, each include respective data dynamically extensible file systems (DEFSs) (e.g., data DEFS **620a** and data DEFS **620b**) and respective log DEFSs (e.g., log DEFS **625a** and log DEFS **625b**). In

general, data DEFSs may be used for persisting data on behalf of clients (e.g., client **180**), whereas log DEFSs may be used to maintain an operation log or journal of certain storage operations within the journaling storage media that have been performed since the last CP.

[0121] It should be noted that while for simplicity only two nodes, which may be configured as part of an HA pair for fault tolerance and nondisruptive operations, are shown in the illustrative cluster depicted in FIG. 6A, there may be one or more additional nodes in a given cluster. For example, there may be multiple HA pairs within a cluster (or a data pod of the cluster, which may represent a mechanism to limit the fault domain). As such, the description of this two-node cluster should be taken as illustrative only. Furthermore, while in some examples HA may be achieved by defining pairs of nodes within a cluster as HA partners (e.g., with one node designated as the primary node and the other designated as the secondary), in alternative examples any other node within a cluster may be allowed to step in after a failure of a given node without defining HA pairs.

[0122] As discussed above, one or more volumes (e.g., volumes **630a-m** and volumes **630n-x**) or LUNs (not shown) may be created by or on behalf of customers for hosting/storing their enterprise application data within respective DEFSs (e.g., data DEFSs **620a** and **620b**).

[0123] While additional data structures may be employed, in this example, each DEFS is shown being associated with respective AA maps (indexed by AA ID) and active maps (indexed by PVBN). For example, log DEFS **625a** may utilize AA map **627a** to track those of the AAs within a global PVBN space **640** of storage pod **645** (which may be analogous to storage pod **145**) that are owned by log DEFS **625a** and may utilize active map **626a** to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; log DEFS **625b** may utilize AA map **627b** to track those of the AAs within the global PVBN space **640** that are owned by log DEFS **625b** and may utilize active map **626b** to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; data DEFS **620a** may utilize AA map **622a** to track those of the AAs within the global PVBN space **640** that are owned by data DEFS **620a** and may utilize active map **621a** to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; and data DEFS **620b** may utilize AA map **622b** to track those of the AAs within the global PVBN space **640** that are owned by data DEFS **620b** and may utilize active map **621b** to track at a PVBN level of granularity which of the PVBNs of its AAs are in use.

[0124] In this example, each DEFS of a given node has visibility and accessibility into the entire global PVBN address space **640** and any AA (except for a predefined super block AA **642**) within the global PVBN address space **640** may be assigned to any DEFS within the cluster. By extension, each node has visibility and accessibility into the entire global PVBN address space **640** via its DEFSs. As noted above, the respective AA maps of the DEFSs define which PVBNs to which the DEFSs have exclusive write access. AAs within the global PVBN space **640** shaded in light gray, such as AA **641a**, can only be written to by node **610a** as a result of their ownership by or assignment to data DEFS **620a**. Similarly AAs within the global PVBN space **640** shaded in dark gray, such as AA **641b**, can only be written to by node **610b** as a result of their ownership by or assignment to data DEFS **620b**.

[0125] Returning to super block **642**, it is part of a super block AA (or super AA). In the context of FIG. 6A, the super AA is the first AA of the storage pod **645**. The super AA is not assigned to any DEFS (as indicated by its lack of shading). The super AA may have an array of DEFS areas which are dedicated to each DEFS and can be indexed by a DEFS ID. The DEFS ID may start at index 1 and in the context of the present example includes four super block and four DEFS label blocks. The DEFS label can act as a RAID label for the DEFS and can be written out of a CP and can store information that needs to be kept outside of the file system. In a pairwise HA configuration, two super blocks and two DEFS label blocks may be used by the hosting node and the other two may be used by the partner node on takeover. Each of these special blocks may have their own separate stripes.

[0126] In the context of the present example, it is assumed after establishment of the disaggregated storage within the storage pod **645** and after the original assignment of ownership of AAs to data DEFS **620a** and data DEFS **620b**, some AAs have been transferred from data DEFS **620a** to data DEFS **620b** and/or some AAs have been transferred from data DEFS **620b** to data DEFS **620a**. As such, the different shades of grayscale of entries within the AA maps are intended to represent potential caching that may be performed regarding ownership of AAs owned by other DEFSs in the cluster. For example, assuming ownership of a partial AA has been transferred from data DEFS **620a** to data DEFS **620b** as part of an ownership change performed in support of space balancing, when data DEFS **620a** would like to free a given PVBN (e.g., when the given PVBN is no longer referenced by data DEFS **620a** as a result of data deletion or otherwise), data DEFS **620a** should send a request to free the PVBN to the new owner (in this case, data DEFS **620b**). This is due to the fact that in various embodiments, only the current owner of a particular AA is allowed to perform any modify operations on the particular AA. Additional details regarding space balancing (AA movement) and changing of AA ownership is provided below with reference to FIG. 7.

[0127] Those skilled in the art will appreciate disaggregation of the storage space as discussed herein can be leveraged for cost-effective scaling of infrastructure. For example, the disaggregated storage allows more applications to share the same underlying storage infrastructure. Given that each DEFS represents an independent file system, the use of multiple of such DEFSs combine to create a cluster-wide distributed file system since all of the DEFSs within a cluster share a global PVBN space (e.g., global PVBN space **640**). This provides the unique ability to independently scale each independent DEFS as well as enables fault isolation and repair in a manner different from existing distributed file systems.

[0128] Additional aspects of FIG. 6A will now be described in connection with a discussion of FIG. 6B, which represents a high-level flow diagram illustrating operations for establishing disaggregated storage within a storage pod (e.g., storage pod **645**). The processing described with reference to FIG. 6B, may be performed by a combination of a file system (e.g., file system **360**) and a RAID system (e.g., RAID system **380**), for example, during or after an initial boot up.

[0129] At block **661**, the storage pod is created based on a set of disks made available for use by the cluster. For

example, job may be executed by a management plane of the cluster to create the storage pod and assign the disks to the cluster. Depending on the particular implementation and the deployment environment (e.g., on-prem versus cloud), the disks may be associated with one or more disk arrays or one or more storage shelves or persistent storage in the form of cloud volumes provided by a cloud provider from a pool of storage devices within a cloud environment. For simplicity, cloud volumes may also be referred to herein as “disks.” The disks may be HDDs or SSDs.

[0130] At block **662**, the storage space of the set of disks may be divided or partitioned into uniform-sized AAs. The set of disks may be grouped to form multiple RAID groups (e.g., RAID group **650a** and **650b**) depending on the RAID level (e.g., RAID 4, RAID 5, or other). Multiple RAID stripes may then be grouped to form individual AAs. As noted above, an AA (e.g., AA **641a** or AA **641b**) may be a large chunk representing one or more GB of storage space and preferably accommodates multiple SSD erase blocks work of data. In one embodiment, the size of the AAs is tuned for the particular file system. The size of the AAs may also take into consideration a desire to reduce the need for performing space balancing so as to minimize the need for internode (e.g., East-West) communications/traffic. In some examples, the size of the AAs may be between about 1 GB to 10 GB. As can be seen in FIG. 6A, dividing the storage pod **645** into AAs allows available storage space associated with any given disk or any RAID group to be use across many/all nodes in the cluster without creating silos of space in each node. For example, at the granularity of an individual AA, available storage space within the storage pod **645** may be assigned to any given node in the cluster (e.g., by way of the given node’s DEFS(s)). For example, in the context of FIG. 6A, AA **641a** and the other AAs shaded in light gray are currently assigned to (or owned by) data DEFS **620a** (which has a corresponding light gray shading). Similarly, AA **641b** and the other AAs shaded in dark gray are currently assigned to (or owned by) data DEFS **620b** (which has a corresponding light gray shading).

[0131] At block **663**, ownership of the AAs is assigned to the DEFSs of the nodes of the cluster. According to one embodiment, an effort may be made to assign group of consecutive AAs to each DEFS. Initially, the distribution of storage space represented by the AAs assigned to each type of DEFS (e.g., data versus log) may be equal or roughly equal. Over time, based on differences in storage consumption by associated workloads, for example, due to differing write patterns, ownership of AAs may be transferred among the DEFSs accordingly.

[0132] As a result, of creating and distributing the disaggregated storage across a cluster in this manner, all disks and all RAID groups can theoretically to be accessed concurrently by all nodes and the issue discussed with reference to FIG. 5 in which the entirety of any given disk and the entirety of any given RAID group is owned by a single node is avoided.

Example AA Movement

[0133] FIG. 7 is a flow diagram illustrating operations for performing AA movement in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. 7 may be performed by a donor DEFS (e.g., data DEFS **620a**, data DEFS **620b**, log DEFS **625a**, or log DEFS **625b** of a storage system (e.g., node **110a**, **110b**,

610a, or 610b) of a distributed storage system (e.g., cluster 100 or a cluster including nodes 610a, 610b, and possibly one or more other nodes).

[0134] At block 710, a space balancing request is received by the donor DEFS. In one embodiment, a heuristics module or AA director, for example, running on a given node of the cluster may be generally responsible for triggering performance of space balancing. For example, the AA director may periodically evaluate AA usage information for all DEFSs in the cluster and based thereon determine donor and recipient DEFSs at a given instance in the cluster. The space balancing request may be received by the donor DEFS, for example, from the AA director via an available internode communication mechanism, various non-limiting examples of which include message passing via on-wire communications and/or the use of one or more persistent message queues (or on-disk message queues), which may make use of the fact that all nodes can read from all disks of a storage pod. For example, one or more persistent message queues may be maintained on behalf of each DEFS in which each DEFS has a message queue to which others can post messages destined for the DEFS. In one embodiment, each DEFS may have an associated inbound queue on which it may receive messages sent by another DEFS in the cluster and an associated outbound queue on which it may post messages intended for delivery to another DEFS in the cluster.

[0135] In the context of the present example, the source of the space balancing request and the nature of the internode communication mechanism through which the space balancing request is received is immaterial and may simply be treated as respective black boxes. The general assumption made in this example is that the space balancing request does not specify the identifiers (IDs) of the particular AAs to be transferred, but rather includes an indication regarding a number of AAs (e.g., 1 or more) to be transferred to a specified recipient DEFS. The space balancing request may also specify a quality of the one or more AAs to be transferred to the recipient DEFS and a RAID group (or AA size) from which the one or more AAs are to be selected. In some examples, RAID groups may be of different widths, which results in AAs of different sizes, so it is helpful to receive either a RAID group ID or the desired AA size as part of the space balancing request. As a non-limiting example, the initiator of the AA movement may specify a set of partial AAs having less than a particular threshold (e.g., 10%, 20%, 30%, etc.) of its storage space utilized be transferred to the recipient DEFS from RAID group 1. Alternatively, the quality of the one or more AAs may be specified by identifying a particular AA bin or bucket (e.g., AAs having 10% to 15% used space). To facilitate selection of AAs from a given AA bin or bucket, the storage system may track which AAs are associated with each AA bin or bucket. Depending on the particular implementation, a given AA may be reevaluated and recategorized, for example, responsive to a threshold number of PVBN allocations and/or frees.

[0136] At block 720, the donor DEFS selects one or more AAs to be donated to the recipient based on criteria included in the space balancing request. The criteria may include the number of AAs and one or more of (i) a specified RAID group ID or AA size and (ii) the desired AA quality. As noted above, in some examples, space balancing (or AA movement) can be used to balance (i) free space, (ii) used space,

and/or (iii) AA quality, for example, using a specified AA bin or bucket from which the one or more AAs to be donated are to be selected by the donor DEFS. The benefit of balancing free space within the cluster should be clear on its face. With respect to balancing of used space, it will be appreciated that such balancing is beneficial because some operations scale with the used space (e.g., freeing of blocks, performance of certain background scanning, etc.). Additionally, it may be helpful to balance AA quality so that the nodes of the cluster are all generally using AAs with similar quality. In one embodiment, if insufficient AAs owned by the donor DEFS satisfy the space balancing request, a smaller number may be selected so as to move whatever it can to the recipient DEFS.

[0137] At block 730, memory-resident AA ownership information maintained by the donor DEFS is updated. For example, the donor DEFS may remove those AAs selected for donation in block 720 from an in-memory AA ownership hash. Additionally, the donor DEFS may remove the AAs from an AA cache, if implemented, to ensure these AAs are no longer used by processes/workflows associated with the donor DEFS. The processing performed in blocks 730 and 740, among other things, seek to prevent further allocations of PVBNs from the selected AAs and prevent PVBN frees and reference count updates. To be sure, however, that the metafile data being transferred as part of the AA movement process includes the latest information, in one example, the metadata file content may be frozen (or quarantined) by waiting until the next CP after updating the in-memory structures before proceeding to make persistent changes. For example, in the event that any changes are made to a given metafile (e.g., an active map, such as active map 621a) in the current CP before the AA ownership information is changed, it is desirable to wait for the CP to write allocate these blocks. In the next CP, changes to the metafile data should be prevented completely. So, at that time buffer tree PVBNs for the AA(s) at issue will be stable.

[0138] At block 740, persistent AA ownership information is updated. In one embodiment, an AA owner file is maintained for each DEFS in the cluster and identifies the AAs (e.g., by AA ID) that are owned by the DEFS. In such an embodiment, updating the persistent AA ownership information involves removing the IDs of the AAs to be donated from the AA owner file of the donor DEFS.

[0139] At block 750, local metafile data (or information associated therewith) for the selected one or more AAs (assuming they are partial AAs) is copied into a payload of an AA movement message (or payload(s) of respective messages) to be transferred to the recipient DEFS. As noted above, the metadata information may track information such as usage information (e.g., which PVBNs of the AAs owned by a given DEFS are in use), PVBN counts per AA (e.g., total used blocks and shared references to blocks), and/or other flags. The shared references to blocks or reference count information may represent a number of other references that exist to a given L0 block resulting from performance of deduplication. Depending on the particular implementation, some metafiles may contain metadata for a single AA and some may contain metadata for multiple AAs. As noted above, when an AA is an empty or free AA (no PVBNs are in use), then there is no metadata information to be transferred along with the transfer of the AA; however, when the AA is a partial AA (at least one PVBN is in use), there will be metadata information associated with the AA that should be transferred to the recipient DEFS. In one embodi-

ment, a single message may be used to transfer multiple empty AAs from the same RAID group. Depending on message size limitations (e.g., a message limited to 4 KB so as to correspond with the size of an L0 block), each partial AA and its associated metadata may be sent to the recipient in separate messages. A non-limiting example of a message format for transferring AAs and their associated metadata information is described below with reference to FIG. 9.

[0140] A simple but inefficient approach to transferring metafile data for a given AA is to copy the underlying metafile data (e.g., stored in L0 blocks on disk) into the payload of the message. Other more efficient approaches involving the transfer of PVBNs (or locations of the underlying metafile data) are described below with reference to FIG. 9 and FIGS. 10A-B.

[0141] At block 760, local metafiles may be holepunched. In one embodiment, the L1 entries for the affected L0 blocks being transferred to the recipient DEFS are holepunched on the donor DEFS. For example, in the case of the un-optimized approach, in which the metafile L0 contents is copied into the message payload, the L1 entry may be holepunched or the contents of the L0 just copied may be zeroed out.

[0142] Assuming use of the “first optimization” of the two metafile data transfer optimization described below with reference to FIG. 10A, those of the bitmap PVBNs (e.g., active map and refcount map) of the local metafiles for the AA(s) being transferred to the recipient DEFS are holepunched. According to one embodiment, this involves zeroing the L1 entries of the local metafiles and invalidating the L0 buffer or in-memory block cache instance, but not freeing the L0 PVBN so as to allow it to be used by the recipient DEFS. The L0 PVBN can be freed (e.g., caused to be freed via a remote PVBN free mechanism) by the recipient DEFS after it overwrites the corresponding L0 file block number.

[0143] Further holepunching that may be performed in connection with the other (the “second optimization”) of the two metafile data transfer optimizations is described below with reference to FIG. 10B. For example, in the second optimization, the L1 PVBNs are copied out of an L2 block, then the L1 entries are holepunched to generate a new L1 PVBN on the donor DEFS that doesn’t know anything about the affected L0 (they are holes). As such, in the second optimization, there are 2 L1 PVBNs for the same logical L1: (i) the previous value that contains PVBNs for everything the recipient DEFS needs and (ii) the new value that has the affected regions wiped out (for continued use on the donor DEFS).

[0144] At block 770, local file system information is updated. For example, based on the donation of the one or more AAs and their respective PVBN counts and usage information, the total PVBNs available to the local file system and usage information may be adjusted accordingly.

[0145] At block 780, the AA movement message is caused to be transferred to the recipient DEFS. For example, the message may be added to an outgoing persistent message queue specifying a destination as the recipient DEFS.

[0146] In some examples, blocks 710 to 730 are idempotent and can be repeated in the event of a crash. With respect to blocks 740 to 780, while extra on-disk state may be used to handle recovery, these blocks should generally be performed during the same CP to ensure crash consistency.

[0147] With reference to the recipient DEFS, upon receipt of AA movement message(s), it essentially performs the same steps but in reverse order. For example, the recipient DEFS may extract the message from its incoming persistent message queue, update its local file system information based on the newly received AAs, transfer, for example, by reference (e.g., via PVBN indirection), the metafile data into a local buffer tree on the second node, update its persistent AA ownership information, and update memory-resident AA ownership information maintained by the recipient DEFS. Upon completion of processing of all AA movement messages by the recipient DEFS for a set of AAs being transferred by the donor DEFS to the recipient DEFS, the metafile data for the set of AAs when linked into a file system operable on the second node hosting the recipient DEFS provides the second node with full metadata information about the set of AAs.

[0148] Of note, as independent file systems, the donor DEFS and the recipient DEFS have independent CP counts. As such, when data is moved from the donor DEFS to the recipient DEFS, the CP count associated with the data is that of the donor DEFS and it might be behind or ahead of the recipient DEFS’s CP count. Therefore, in one example, before the recipient DEFS process such a movement, on the recipient DEFS the CP count is updated to be greater than the CP associated with the data being transferred to it. For example, the recipient DEFS may update its CP count to 1 more than the CP associated with the data being transferred to it because the transferred PVBNs point to on-disk data that includes the CP count of the source filesystem.

[0149] While in the context of the flow diagrams of FIGS. 6B and 7 a number of enumerated blocks are included, it is to be understood that examples may include additional blocks before, after, and/or in between the enumerated blocks. Similarly, in some examples, one or more of the enumerated blocks may be omitted and/or performed in a different order. Also, it is to be noted that more or fewer metafiles for AAs might also be transferred in different implementations.

Example Buffer Tree

[0150] FIG. 8 is a block diagram conceptually illustrating a portion of a buffer tree 800 containing information regarding a metafile 860 in accordance with an embodiment of the present disclosure. The buffer tree 800 may generally correspond to the file system layout shown and described with reference to FIG. 4. In this example, however, only the last layers of indirect blocks (e.g., inode file data block 830 and L1 blocks 840a-n) are shown. In this example, inode file data block 830 (which may also be referred to herein as an L2 block) is shown containing multiple L1 PVBNs (i.e., L1 PVBNs 831a-n, each of which may also be referred to individually as an L2 entry or collectively as L2 entries) that contain the location of or a pointer to respective L1 blocks of the metafile 860. Each of these L1 blocks is further shown as containing multiple L0 PVBNs (L0 PVBNs 841a-m, each of which may also be referred to individually as an L1 entry or collectively as L1 entries) that contain the location of or a pointer to respective L0 blocks (e.g., L0 block 850a-m) on disk that contain metafile data.

Example Message Format

[0151] FIG. 9 is a block diagram conceptually illustrating a message format 900 that may be used in connection with

performing AA movement in accordance with an embodiment of the present disclosure. In this example, it is assumed a single AA is transferred from the donor DEFS to the recipient DEFS via a single AA movement message in order to ensure the entire message fits within an L0 block, for example of 4 KB. In other examples, a single message may be used to transfer multiple AAs. For example, multiple free AAs (which need not be transferred with associated metafile data) may be transferred in a single message.

[0152] In the context of the present example, the message format **900** is shown including a message header, an AA ID and an ID of the destination (recipient) DEFS, AA information (e.g., the total number of used/free blocks in the AA, storage efficiency information, and number of disks in the RAID group), and for each disk of the RAID group across which the AA is striped, one or more active map L1 PVBNs **910a-n**, one or more reference count L1 PVBNs **920a-n**, one or more space map L0 PVBNs **930a-n**, and other optional metafile data **940a-n** (e.g., relating to other files for which PVBNs per disk may be sent). In the illustrated embodiment, the space map metafile data may be transferred by reference via one or more L0 PVBNs under the assumption, for example, that the space map metafile includes metafile data for only a single AA.

Metafile Data Transfer Approaches

[0153] As noted above, there are various approaches that may be used to transfer the metafile data for a given AA from a donor DEFS to a recipient DEFS. In one embodiment, no optimization may be employed and the donor DEFS may simply copy the L0 contents as needed into the message payload and recipient DEFS copies it out of the message payload into its local file system metadata. Then, the L0 contents being transferred may be zeroed out as needed or holepunch and subsequently free the PVBNs.

[0154] In one example, a first optimization may be used to transfer metafile data in which the donor DEFS copies the L0 PVBNs into the message payload and the recipient DEFS does the opposite. The donor DEFS holepunches these L1 entries and does not free the PVBNs.

[0155] In another example, a second optimization may be used to transfer metafile data in which the donor DEFS copies L1 PVBNs into the message payload. Then, the donor DEFS holepunches the L1 entries without freeing anything. The donor DEFS continues using a newly allocated L1 PVBN that has 0s/holes for these entries, while the recipient DEFS uses the old L1 PVBN that has PVBNs for the L0s. The recipient DEFS copies these L0 PVBNs out of the message payload and then frees this L1 PVBN back on the donor DEFS (or wherever it lives). The L0 PVBNs can be freed in accordance with the traditional way in which the file system (e.g., the WAFL file system) would normally do so, for example, when the file block number (FBN) is later overwritten.

Example First Optimization

[0156] FIG. 10A is a block diagram conceptually illustrating a first optimization for transferring metafile data in accordance with an embodiment of the present disclosure. As noted above, one approach, albeit inefficient, for transferring metafile data for a given AA that is being transferred from a donor DEFS to a recipient DEFS, involves copying the underlying metafile data (e.g., stored in L0 blocks on

disk) into the payload of a message (e.g., an AA movement message). One example of a more efficient approach involves the transfer of PVBNs (directly or indirectly providing locations) of the underlying metafile data. As will be appreciated, transferring data by reference in this matter allows for creation of a more compact message. Furthermore, copying of PVBNs is faster and more efficient than copying the underlying metafile data and can be performed in constant time. For example, in a case in which there is multiple GB of metafile data, it is clearly faster to copy the location of the data instead of the data.

[0157] In the context of the present example, the first optimization involves copying L0 PVBNs (e.g., L0 PVBN **1011a-n**) from a buffer tree (e.g., buffer tree **800**) containing information regarding the metafile at issue into a message **1010**. The message **1010** may be analogous to the AA movement message described with reference to FIG. 7 and/or an AA movement message having the format as described with reference to FIG. 9).

Example Second Optimization

[0158] FIG. 10B is a block diagram conceptually illustrating a second optimization for transferring metafile data in accordance with an embodiment of the present disclosure. In the context of the present example, the second optimization involves copying one or more L1 PVBNs (e.g., L1 PVBN **1031a-n**) from a buffer tree (e.g., buffer tree **800**) containing information regarding the metafile at issue into a message **1030**. The message **1030** may be analogous to the AA movement message described with reference to FIG. 7 and/or an AA movement message having the format as described with reference to FIG. 9).

[0159] As noted above, in some examples, a given metafile may contain metafile data for a single AA or may contain metafile data for multiple AAs. When transferring metafile data from the donor DEFS to the recipient DEFS by reference (e.g., indirectly via the use of L1 PVBNs) in accordance with the second optimization, the L0 PVBNs in the donor DEFS's buffer tree corresponding to metafile data of the AAs being transferred to the recipient DEFS may be holepunched to avoid inadvertent usage of the transferred metafile data by processes/workflows associated with the donor DEFS. In this example, L0 PVBNs **1041a-1041m** (shown with a gray background), which correspond to the metafile data for the AA at issue are holepunched locally, and L0 PVBNs **1041m+1** to **1041n** (shown with a white background) are maintained. As noted above, the holepunched L0 PVBNs should not be freed by the donor DEFS. Rather, these L0 PVBNs (or L1 entries) can be freed by the recipient DEFS after these L0 PVBNs have been linked into the recipient DEFS and have been subsequently overwritten by the recipient DEFS. As additionally noted above, nor should the L1 PVBN be freed by the donor DEFS as part of writing the newly punched holes into it at a new PVBN, as this L1 PVBN will be freed by the recipient DEFS after the L1 entries of the transferred L1 PVBN(s) have been copied into the file system of the recipient DEFS.

[0160] Depending on the particular implementation, the first optimization or the second optimization may be selectively employed to build an AA movement message by the donor DEFS based on the number of PVBNs involved, based on whether the metafile at issue contains metafile data for a single AA or multiple AA, and/or based on other factors.

[0161] Embodiments of the present disclosure include various steps, which have been described above. The steps may be performed by hardware components or may be embodied in machine-executable instructions, which may be used to cause one or more processing resources (e.g., one or more general-purpose or special-purpose processors) programmed with the instructions to perform the steps. Alternatively, depending upon the particular implementation, various steps may be performed by a combination of hardware, software, firmware and/or by human operators.

[0162] Embodiments of the present disclosure may be provided as a computer program product, which may include a non-transitory machine-readable storage medium embodying thereon instructions, which may be used to program a computer (or other electronic devices) to perform a process. The machine-readable medium may include, but is not limited to, fixed (hard) drives, magnetic tape, floppy diskettes, optical disks, compact disc read-only memories (CD-ROMs), and magneto-optical disks, semiconductor memories, such as ROMs, PROMs, random access memories (RAMs), programmable read-only memories (PROMs), erasable PROMs (EPROMs), electrically erasable PROMs (EEPROMs), flash memory, magnetic or optical cards, or other type of media/machine-readable medium suitable for storing electronic instructions (e.g., computer programming code, such as software or firmware).

[0163] Various methods described herein may be practiced by combining one or more non-transitory machine-readable storage media containing the code according to embodiments of the present disclosure with appropriate special purpose or standard computer hardware to execute the code contained therein. An apparatus for practicing various embodiments of the present disclosure may involve one or more computers (e.g., physical and/or virtual servers) (or one or more processors (e.g., processors **222a-b**) within a single computer) and storage systems containing or having network access to computer program(s) coded in accordance with various methods described herein, and the method steps associated with embodiments of the present disclosure may be accomplished by modules, routines, subroutines, or sub-parts of a computer program product.

[0164] The term “storage media” as used herein refers to any non-transitory media that store data or instructions that cause a machine to operation in a specific fashion. Such storage media may comprise non-volatile media or volatile media. Non-volatile media includes, for example, optical, magnetic or flash disks, such as storage device (e.g., local storage **230**). Volatile media includes dynamic memory, such as main memory (e.g., memory **224**). Common forms of storage media include, for example, a flexible disk, a hard disk, a solid state drive, a magnetic tape, or any other magnetic data storage medium, a CD-ROM, any other optical data storage medium, any physical medium with patterns of holes, a RAM, a PROM, and EPROM, a FLASH-EPROM, NVRAM, any other memory chip or cartridge.

[0165] Storage media is distinct from but may be used in conjunction with transmission media. Transmission media participates in transferring information between storage media. For example, transmission media includes coaxial cables, copper wire and fiber optics, including the wires that comprise bus (e.g., system bus **223**). Transmission media can also take the form of acoustic or light waves, such as those generated during radio-wave and infra-red data communications.

[0166] Various forms of media may be involved in carrying one or more sequences of one or more instructions to the one or more processors for execution. For example, the instructions may initially be carried on a magnetic disk or solid state drive of a remote computer. The remote computer can load the instructions into its dynamic memory and send the instructions over a telephone line using a modem. A modem local to the computer system can receive the data on the telephone line and use an infra-red transmitter to convert the data to an infra-red signal. An infra-red detector can receive the data carried in the infra-red signal and appropriate circuitry can place the data on bus. Bus carries the data to main memory (e.g., memory **224**), from which the one or more processors retrieve and execute the instructions. The instructions received by main memory may optionally be stored on storage device either before or after execution by the one or more processors.

[0167] All examples and illustrative references are non-limiting and should not be used to limit the applicability of the proposed approach to specific implementations and examples described herein and their equivalents. For simplicity, reference numbers may be repeated between various examples. This repetition is for clarity only and does not dictate a relationship between the respective examples. Finally, in view of this disclosure, particular features described in relation to one aspect or example may be applied to other disclosed aspects or examples of the disclosure, even though not specifically shown in the drawings or described in the text.

[0168] The foregoing outlines features of several examples so that those skilled in the art may better understand the aspects of the present disclosure. Those skilled in the art should appreciate that they may readily use the present disclosure as a basis for designing or modifying other processes and structures for carrying out the same purposes and/or achieving the same advantages of the examples introduced herein. Those skilled in the art should also realize that such equivalent constructions do not depart from the spirit and scope of the present disclosure, and that they may make various changes, substitutions, and alterations herein without departing from the spirit and scope of the present disclosure.

What is claimed is:

1. A method comprising:

providing a storage pod having a group of storage devices containing a plurality of Redundant Array of Independent Disks (RAID) groups, wherein a global physical volume block number (PVCN) space associated with the storage pod is accessible to all nodes of a plurality of nodes of a cluster representing a distributed storage system via their respective dynamically extensible file systems (DEFSs) and wherein storage space associated with the group is partitioned into a plurality of allocation areas (AAs), in which a given AA of the plurality of AAs is owned by a given DEFS of a plurality of DEFSs of the cluster; and

receiving, by a donor DEFS of a first node of the plurality of nodes, a space balancing request indicating (i) a number of AAs to be transferred to a recipient DEFS of a second node of the plurality of nodes and (ii) a quality of the AAs to be transferred;

selecting, by the first node, a set of AAs currently owned by the donor DEFS that satisfy the space balancing request; and

transferring, by the first node, ownership of the set of AAs to the recipient DEFS and metadata information associated with the set of AAs.

2. The method of claim 1, further comprising precluding subsequent use, by the first node, of file system metadata associated with the set of AAs by, for each AA of the set of AAs, hole-punching a PVBN within a level-1 (L1) block pointing to the AA.

3. The method of claim 2, further comprising removing the file system metadata from an in-memory AA cache used by a file system operable on the first node.

4. The method of claim 1, wherein transfer of the metadata information includes sending a message to the recipient DEFS, in which the message includes one or more level-0 (L0) PVBNs, and wherein the one or more L0 PVBNs point to one or more respective L0 blocks containing metafile data for the set of AAs.

5. The method of claim 1, wherein transfer of the metadata information includes sending a message to the recipient DEFS, in which the message includes one or more level-1 (L1) PVBNs that include a plurality of L0 PVBNs pointing to respective L0 blocks, and wherein a first subset of the respective L0 blocks contain metafile data for the set of AAs and a second subset of the respective L0 blocks contain metafile data for one or more other AAs.

6. The method of claim 5, further comprising hole-punching, by the first node, those of the plurality of L0 PVBNs pointing to the first subset of the respective L0 blocks.

7. The method of claim 1, further comprising, prior to said transferring:

updating, by the first node, an in-memory data structure that tracks AAs owned by the donor DEFS to mark the set of AAs as being in an intermediate state of ownership transfer; and

updating, by the first node, a persistent file that tracks the AAs owned by the donor DEFS to indicate the set of AAs are no longer owned by the donor DEFS.

8. The method of claim 1, wherein transfer of the metadata information includes sending a message to the recipient DEFS, in which the message includes one or more PVBNs that (i) represent locations of blocks containing metafile data for the set of AAs or (ii) represent locations of blocks containing further PVBNs that represent the locations of the blocks containing metafile data for the set of AAs.

9. The method of claim 1, wherein the metafile data for the set of AAs when linked into a file system operable on the second node provides the second node with full metadata information about the set of AAs.

10. The method of claim 1, wherein the metadata information is part of a metafile, wherein responsive to said selecting and until completion of said transferring is complete, the metadata information is frozen by precluding access to the metafile by the donor DEFS.

11. A non-transitory machine readable medium storing instructions, which when executed by one or more processing resources of a distributed storage system, cause the distributed storage system to:

provide a storage pod having a group of storage devices containing a plurality of Redundant Array of Independent Disks (RAID) groups, wherein an entirety of a global physical volume block number (PVBN) space associated with the storage pod is accessible to all nodes of a plurality of nodes of a cluster representing

the distributed storage system via their respective dynamically extensible file systems (DEFSs) and wherein storage space associated with the group is partitioned into a plurality of allocation areas (AAs), in which a given AA of the plurality of AAs is owned by a given DEFS of a plurality of DEFSs of the cluster; and

receive, by a donor DEFS of a first node of the plurality of nodes, a space balancing request indicating (i) a number of AAs to be transferred to a recipient DEFS of a second node of the plurality of nodes and (ii) a quality of the AAs to be transferred;

select, by the first node, a set of AAs currently owned by the donor DEFS that satisfy the space balancing request; and

transfer, by the first node, ownership of the set of AAs to the recipient DEFS and metadata information associated with the set of AAs.

12. The non-transitory machine readable medium of claim 11, wherein the instructions further cause the distributed storage system to preclude subsequent use, by the first node, of file system metadata associated with the set of AAs by, for each AA of the set of AAs, hole-punching a PVBN within a level-1 (L1) block pointing to the AA.

13. The non-transitory machine readable medium of claim 11, wherein transfer of the metadata information includes sending a message to the recipient DEFS, in which the message includes one or more level-0 (L0) PVBNs, and wherein the one or more L0 PVBNs point to one or more respective L0 blocks containing metafile data for the set of AAs.

14. The non-transitory machine readable medium of claim 11, wherein transfer of the metadata information includes sending a message to the recipient DEFS, in which the message includes one or more level-1 (L1) PVBNs that include a plurality of L0 PVBNs pointing to respective L0 blocks, and wherein a first subset of the respective L0 blocks contain metafile data for the set of AAs and a second subset of the respective L0 blocks contain metafile data for one or more other AAs.

15. The non-transitory machine readable medium of claim 14, wherein the instructions further cause the distributed storage system to hole-punch, by the first node, those of the plurality of L0 PVBNs pointing to the first subset of the respective L0 blocks.

16. A distributed storage system comprising:

one or more processing resources; and

instructions that when executed by the one or more processing resources cause the distributed storage system to:

provide a storage pod having a group of storage devices containing a plurality of Redundant Array of Independent Disks (RAID) groups, wherein an entirety of a global physical volume block number (PVBN) space associated with the storage pod is accessible to all nodes of a plurality of nodes of a cluster representing the distributed storage system via their respective dynamically extensible file systems (DEFSs) and wherein storage space associated with the group is partitioned into a plurality of allocation areas (AAs), in which a given AA of the plurality of AAs is owned by a given DEFS of a plurality of DEFSs of the cluster; and

receive, by a donor DEFS of a first node of the plurality of nodes, a space balancing request indicating (i) a number of AAs to be transferred to a recipient DEFS of a second node of the plurality of nodes and (ii) a quality of the AAs to be transferred;

select, by the first node, a set of AAs currently owned by the donor DEFS that satisfy the space balancing request; and

transfer, by the first node, ownership of the set of AAs to the recipient DEFS and metadata information associated with the set of AAs.

17. The distributed storage system of claim **16**, wherein the instructions further cause the distributed storage system to preclude subsequent use, by the first node, of file system metadata associated with the set of AAs by, for each AA of the set of AAs, hole-punching a PVBN within a level-1 (L1) block pointing to the AA.

18. The distributed storage system of claim **16**, wherein transfer of the metadata information includes sending a

message to the recipient DEFS, in which the message includes one or more level-0 (L0) PVBNs, and wherein the one or more L0 PVBNs point to one or more respective L0 blocks containing metafile data for the set of AAs.

19. The distributed storage system of claim **16**, wherein transfer of the metadata information includes sending a message to the recipient DEFS, in which the message includes one or more level-1 (L1) PVBNs that include a plurality of L0 PVBNs pointing to respective L0 blocks, and wherein a first subset of the respective L0 blocks contain metafile data for the set of AAs and a second subset of the respective L0 blocks contain metafile data for one or more other AAs.

20. The distributed storage system of claim **19**, wherein the instructions further cause the distributed storage system to hole-punch, by the first node, those of the plurality of L0 PVBNs pointing to the first subset of the respective L0 blocks.

* * * * *